

Verificación de Programas Concurrentes Usando Lógica

Logical Foundations of Concurrent Computation

Jorge A. Pérez
University of Groningen, The Netherlands
j.a.perez [[at]] rug.nl

ECI 2026
(Version of July 27, 2026)

Preambulo

Verificación de Programas Concurrentes

- ▶ Muchos sistemas informáticos son **concurrentes**, y están compuestos de programas que se ejecutan simultáneamente.

Verificación de Programas Concurrentes

- ▶ Muchos sistemas informáticos son **concurrentes**, y están compuestos de programas que se ejecutan simultáneamente.
- ▶ Ejemplos:
 - Sistemas distribuidos y paralelos
 - Arquitecturas basadas en microservicios
 - Go / Erlang / Rust / Akka / Elixir

Verificación de Programas Concurrentes

- ▶ Muchos sistemas informáticos son **concurrentes**, y están compuestos de programas que se ejecutan simultáneamente.
- ▶ Ejemplos:
 - Sistemas distribuidos y paralelos
 - Arquitecturas basadas en microservicios
 - Go / Erlang / Rust / Akka / Elixir
- ▶ Mecanismo de coordinación entre programas: **paso de mensajes**.

Verificación de Programas Concurrentes

- ▶ Muchos sistemas informáticos son **concurrentes**, y están compuestos de programas que se ejecutan simultáneamente.
- ▶ Ejemplos:
 - Sistemas distribuidos y paralelos
 - Arquitecturas basadas en microservicios
 - Go / Erlang / Rust / Akka / Elixir
- ▶ Mecanismo de coordinación entre programas: **paso de mensajes**.
- ▶ Establecer la correctitud de estos programas implica garantizar que las interacciones entre ellos respetan los **protocolos establecidos**.

Verificación de Programas Concurrentes

- ▶ Muchos sistemas informáticos son **concurrentes**, y están compuestos de programas que se ejecutan simultáneamente.
- ▶ Ejemplos:
 - Sistemas distribuidos y paralelos
 - Arquitecturas basadas en microservicios
 - Go / Erlang / Rust / Akka / Elixir
- ▶ Mecanismo de coordinación entre programas: **paso de mensajes**.
- ▶ Establecer la correctitud de estos programas implica garantizar que las interacciones entre ellos respetan los **protocolos establecidos**.
- ▶ Una tarea crucial, pero también compleja: necesitamos técnicas de verificación capaces de analizar **estructuras de comunicación** sofisticadas.

Verificación de Programas Concurrentes Usando Lógica

Este Curso

- ▶ Técnicas fundamentales para la verificación para programas concurrentes.
- ▶ **Enfasis:** Conceptos de la **lógica** para el análisis y verificación de programas.

Verificación de Programas Concurrentes Usando Lógica

Este Curso

- ▶ Técnicas fundamentales para la verificación para programas concurrentes.
- ▶ **Enfasis:** Conceptos de la **lógica** para el análisis y verificación de programas.

Ideas Principales

- ▶ Describir los protocolos de comunicación usando **sistemas de tipos**.
 - ▶ Especificar los programas concurrentes con un **lenguaje formal de procesos**.
 - ▶ Los tipos aseguran que los procesos **usan sus recursos correctamente**.
- Uso adecuado de recursos = ejecución correcta de protocolos.

Verificación de Programas Concurrentes Usando Lógica

Este Curso

- ▶ Técnicas fundamentales para la verificación para programas concurrentes.
- ▶ **Enfasis:** Conceptos de la **lógica** para el análisis y verificación de programas.

Ideas Principales

- ▶ Describir los protocolos de comunicación usando **sistemas de tipos**.
 - ▶ Especificar los programas concurrentes con un **lenguaje formal de procesos**.
 - ▶ Los tipos aseguran que los procesos **usan sus recursos correctamente**.
- Uso adecuado de recursos = ejecución correcta de protocolos.

Correspondencia de Curry-Howard

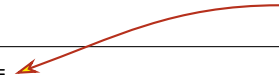
Demostrar un teorema y escribir un programa correcto son la **misma actividad** desde perspectivas distintas.

Ejemplo: Un Programa con Paso de Mensajes

```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```

Ejemplo: Un Programa con Paso de Mensajes

Una función con dos parámetros (dos canales)



```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```

Ejemplo: Un Programa con Paso de Mensajes

```
peter c1 c2 =
```

```
  let (x, d1) = receive c1 in
```

```
  let d2 = send (41) c2 in
```

```
  wait d1;
```

```
  close d2;
```

```
  ()
```

Recibe un mensaje en c1, continúa en d1



Ejemplo: Un Programa con Paso de Mensajes

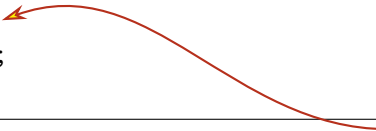
```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```



Envía '41' en c2, continúa en d2

Ejemplo: Un Programa con Paso de Mensajes

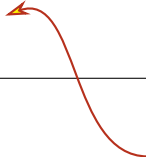
```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```



Espera confirmación para cerrar canal d1

Ejemplo: Un Programa con Paso de Mensajes

```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```



Envía mensaje para cerrar canal d2

Ejemplo: Un Programa con Paso de Mensajes

```
peter c1 c2 =  
  let (x, d1) = receive c1 in  
  let d2 = send (41) c2 in  
  wait d1;  
  close d2;  
  ()
```



Retorna valor de tipo unit

Ejemplo: Dos Programas con Paso de Mensajes

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

Ejemplo: Dos Programas con Paso de Mensajes

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

El programa respeta su protocolo ✓

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

Ejemplo: Dos Programas con Paso de Mensajes

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

El programa respeta su protocolo ✓

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

El programa también respeta su protocolo ✓

Ejemplo: Interacción de Programas

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

```
main =  
  let (c1, c1dual) = new @T () in  
  let (c2, c2dual) = new @T () in  
  fork (miles c1dual c2);  
  peter c1 c2dual
```

Es 'main' correcto?

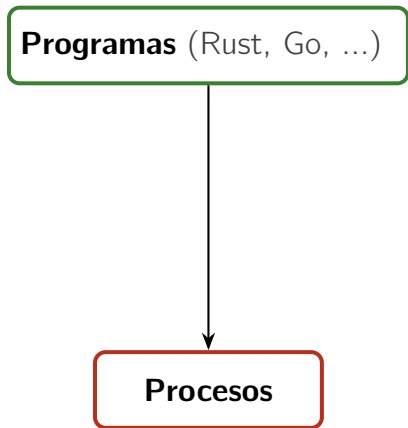


Verificación de Programas usando Sistemas de Tipos

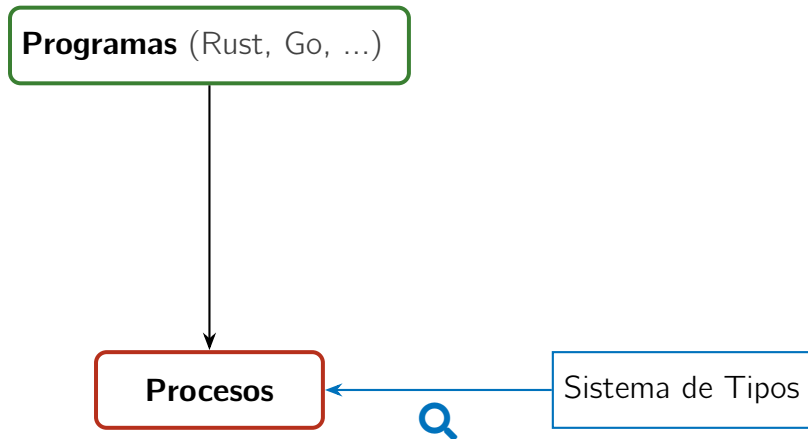
Programas (Rust, Go, ...)

Procesos

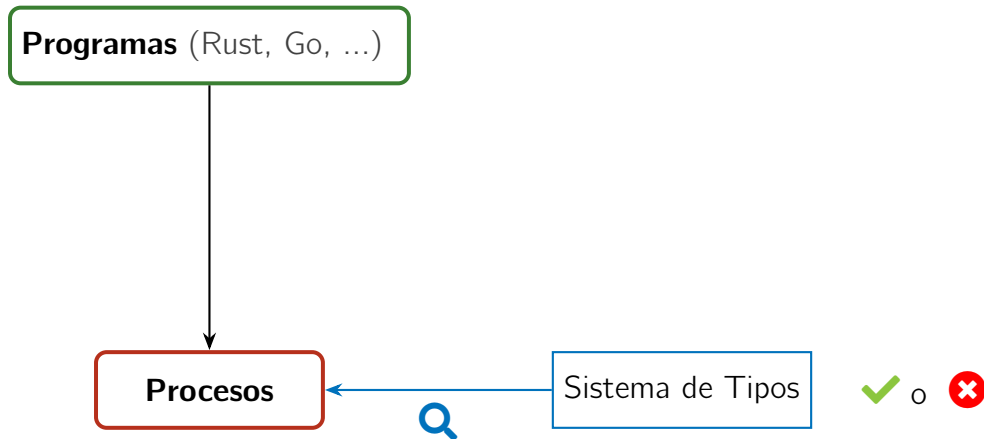
Verificación de Programas usando Sistemas de Tipos



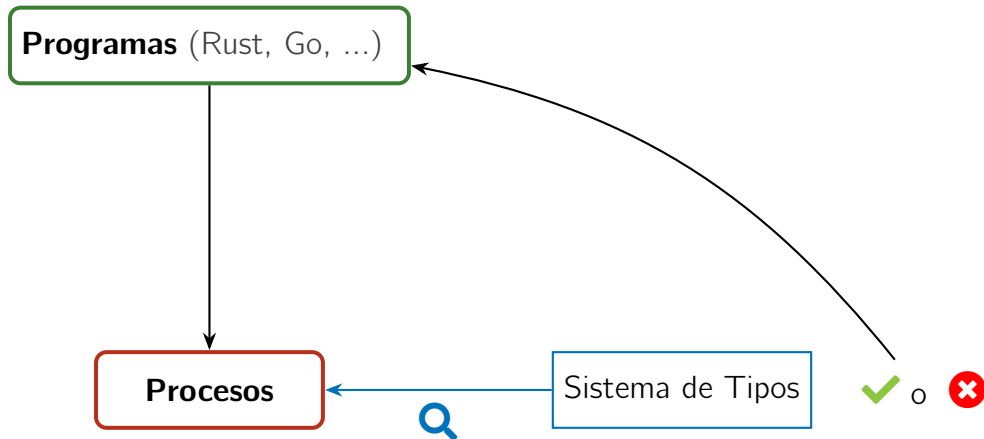
Verificación de Programas usando Sistemas de Tipos



Verificación de Programas usando Sistemas de Tipos



Verificación de Programas usando Sistemas de Tipos



Un Lenguaje Formal para Concurrencia: Cálculos de Procesos

Asumiendo un conjunto de **nombres** $x, y, z, \dots$ (usados para representar **canales**), definimos un lenguaje formal de **procesos** (denotados $P, Q, \dots$):

Un Lenguaje Formal para Concurrencia: Cálculos de Procesos

Asumiendo un conjunto de **nombres** $x, y, z, \dots$ (usados para representar **canales**), definimos un lenguaje formal de **procesos** (denotados $P, Q, \dots$):

$$\begin{array}{l} P, Q ::= \bar{x}\langle y \rangle.P \quad \textbf{envia} \text{ mensaje } y \text{ en el canal } x, \text{ continúa con } P \\ \quad \mid x(y).P \quad \textbf{recibe} \text{ un mensaje } z \text{ en el canal } x, \text{ continúa con } P\{z/y\} \end{array}$$

Un Lenguaje Formal para Concurrencia: Cálculos de Procesos

Asumiendo un conjunto de **nombres** $x, y, z, \dots$ (usados para representar **canales**), definimos un lenguaje formal de **procesos** (denotados $P, Q, \dots$):

$P, Q ::=$	$\bar{x}\langle y \rangle.P$	envía mensaje y en el canal x , continúa con P
	$x(y).P$	recibe un mensaje z en el canal x , continúa con $P\{z/y\}$
	$\bar{x}\langle \rangle.P$	envía una señal para cerrar el canal x , continúa como P
	$x().P$	recibe señal para cerrar el canal x , continúa como P

Un Lenguaje Formal para Concurrencia: Cálculos de Procesos

Asumiendo un conjunto de **nombres** $x, y, z, \dots$ (usados para representar **canales**), definimos un lenguaje formal de **procesos** (denotados $P, Q, \dots$):

$P, Q ::=$	$\bar{x}\langle y \rangle.P$	envía mensaje y en el canal x , continúa con P
	$x(y).P$	recibe un mensaje z en el canal x , continúa con $P\{z/y\}$
	$\bar{x}\langle \rangle.P$	envía una señal para cerrar el canal x , continúa como P
	$x().P$	recibe señal para cerrar el canal x , continúa como P
	$P \mid Q$	conurrencia : P y Q se ejecutan en paralelo
	$(\nu x)P$	restricción : el canal x es local (privado) en P
	0	cero : el proceso que no hace nada

Un Lenguaje Formal: Cálculos de Procesos

Sintaxis de procesos:

$$P, Q ::= \bar{x}\langle y \rangle.P \mid x(y).P \mid \bar{x}\langle \rangle.P \mid x().P \mid P \mid Q \mid (\nu x)P \mid 0$$

Cómo expresar interacción?

Un Lenguaje Formal: Cálculos de Procesos

Sintaxis de procesos:

$$P, Q ::= \bar{x}\langle y \rangle.P \mid x(y).P \mid \bar{x}\langle \rangle.P \mid x().P \mid P \mid Q \mid (\nu x)P \mid 0$$

Cómo expresar interacción?

La **relación de reducción** formaliza la comunicación entre procesos concurrentes.

La definición incluye:

$$\bar{x}\langle z \rangle.P \mid x(y).Q \longrightarrow P \mid Q\{z/y\}$$

$$\bar{x}\langle \rangle.P \mid x().Q \longrightarrow P \mid Q$$

Un Lenguaje Formal: Cálculos de Procesos

Sintaxis de procesos:

$$P, Q ::= \bar{x}\langle y \rangle.P \mid x(y).P \mid \bar{x}\langle \rangle.P \mid x().P \mid P \mid Q \mid (\nu x)P \mid 0$$

Cómo expresar interacción?

La **relación de reducción** formaliza la comunicación entre procesos concurrentes.

La definición incluye:

$$\bar{x}\langle z \rangle.P \mid x(y).Q \longrightarrow P \mid Q\{z/y\}$$

$$\bar{x}\langle \rangle.P \mid x().Q \longrightarrow P \mid Q$$

Intuición:

Acciones que ocurren en paralelo (y en el mismo canal) pueden dar lugar a una sincronización. Las acciones deben ser complementarias (input / output).

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

3) $\bar{x}\langle \rangle.x().P$

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

3) $\bar{x}\langle \rangle.x().P$

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

no hay reducción: los canales son diferentes

3) $\bar{x}\langle \rangle.x().P$

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

no hay reducción: los canales son diferentes

3) $\bar{x}\langle \rangle.x().P$

no hay reducción: las acciones son secuenciales

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

no hay reducción: los canales son diferentes

3) $\bar{x}\langle \rangle.x().P$

no hay reducción: las acciones son secuenciales

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

hay dos reducciones

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

no hay reducción: los canales son diferentes

3) $\bar{x}\langle \rangle.x().P$

no hay reducción: las acciones son secuenciales

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

hay dos reducciones

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

hay una reducción: el canal z es un mensaje!

Ejercicio: Cuáles procesos pueden reducir (interactuar)?

1) $x(z).P \mid x(u).Q$

no hay reducción:

las acciones no son complementarias

2) $\bar{x}\langle z \rangle.P \mid z(u).Q$

no hay reducción: los canales son diferentes

3) $\bar{x}\langle \rangle.x().P$

no hay reducción: las acciones son secuenciales

4) $\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q$

hay dos reducciones

5) $\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1$

hay una reducción: el canal z es un mensaje!

Veamos (4) y (5) en detalle:

$$\begin{aligned}\bar{x}\langle v_1 \rangle.x(z).P \mid x(y).\bar{x}\langle v_2 \rangle.Q &\longrightarrow x(z).P \mid \bar{x}\langle v_2 \rangle.Q\{v_1/y\} \\ &\longrightarrow P\{v_2/z\} \mid Q\{v_1/y\}\end{aligned}$$

$$\bar{x}\langle z \rangle.0 \mid x(y).y(u).Q_1 \longrightarrow 0 \mid z(u).Q_1\{z/y\}$$

Un Lenguaje Formal: Cálculos de Procesos

Usando procesos, el protocolo de comunicación del programa `main` se puede expresar de forma formal y compacta:

$$(\nu c)(\nu d)(c(x).\bar{d}\langle 41 \rangle.c().\bar{d}\langle \rangle.0$$

|

$$d(x).\bar{c}\langle 42 \rangle.d().\bar{c}\langle \rangle.0)$$

Un Lenguaje Formal: Cálculos de Procesos

Usando procesos, el protocolo de comunicación del programa `main` se puede expresar de forma formal y compacta:

$$(\nu c)(\nu d)(c(x).\bar{d}\langle 41 \rangle.c().\bar{d}\langle \rangle.0$$
$$|$$
$$d(x).\bar{c}\langle 42 \rangle.d().\bar{c}\langle \rangle.0)$$

Proceso de peter



Un Lenguaje Formal: Cálculos de Procesos

Usando procesos, el protocolo de comunicación del programa `main` se puede expresar de forma formal y compacta:

$$(\nu c)(\nu d)(c(x).\bar{d}\langle 41 \rangle.c().\bar{d}\langle \rangle.0$$

|

$$d(x).\bar{c}\langle 42 \rangle.d().\bar{c}\langle \rangle.0)$$

Proceso de peter

Proceso de miles

Un Lenguaje Formal: Cálculos de Procesos

Usando procesos, el protocolo de comunicación del programa main se puede expresar de forma formal y compacta:

$$(\nu c)(\nu d)(c(x).\bar{d}\langle 41 \rangle.c().\bar{d}\langle \rangle.0$$

|

$$d(x).\bar{c}\langle 42 \rangle.d().\bar{c}\langle \rangle.0)$$

Proceso de peter

Proceso de miles

El análisis del sistema completo nos permite detectar una dependencia circular entre los dos sub-procesos: el programa no puede reducir y llega a un **deadlock**.

Un Lenguaje Formal: Cálculos de Procesos

Usando procesos, el protocolo de comunicación del programa main se puede expresar de forma formal y compacta:

$(\nu c)(\nu d)(c(x).\bar{d}\langle 41 \rangle.c().\bar{d}\langle \rangle.0$

|

$d(x).\bar{c}\langle 42 \rangle.d().\bar{c}\langle \rangle.0)$

Proceso de peter

Proceso de miles

El análisis del sistema completo nos permite detectar una dependencia circular entre los dos sub-procesos: el programa no puede reducir y llega a un **deadlock**.

Plan del Curso

1. **Session types:** Especificaciones precisas de protocolos para canales.

Plan del Curso

1. **Session types:** Especificaciones precisas de protocolos para canales.
2. **Linear Logic (LL):** Una lógica para el uso de recursos computacionales.

Plan del Curso

1. **Session types**: Especificaciones precisas de protocolos para canales.
2. **Linear Logic (LL)**: Una lógica para el uso de recursos computacionales.
3. La **correspondencia de Curry-Howard** para procesos concurrentes:
 - proposiciones en LL $\iff$ session types
 - pruebas en LL $\iff$ procesos concurrentes
 - simplificación de pruebas $\iff$ sincronización entre procesos

Plan del Curso

1. **Session types**: Especificaciones precisas de protocolos para canales.
2. **Linear Logic (LL)**: Una lógica para el uso de recursos computacionales.
3. La **correspondencia de Curry-Howard** para procesos concurrentes:
 - proposiciones en LL $\iff$ session types
 - pruebas en LL $\iff$ procesos concurrentes
 - simplificación de pruebas $\iff$ sincronización entre procesos
4. La correspondencia usando la **versión clásica** de LL.

Plan del Curso

1. **Session types**: Especificaciones precisas de protocolos para canales.
 2. **Linear Logic (LL)**: Una lógica para el uso de recursos computacionales.
 3. La **correspondencia de Curry-Howard** para procesos concurrentes:
 - proposiciones en LL $\iff$ session types
 - pruebas en LL $\iff$ procesos concurrentes
 - simplificación de pruebas $\iff$ sincronización entre procesos
 4. La correspondencia usando la **versión clásica** de LL.
 5. Comunicación asíncrona y deadlock freedom.
-

Plan del Curso

1. **Session types**: Especificaciones precisas de protocolos para canales.
2. **Linear Logic (LL)**: Una lógica para el uso de recursos computacionales.
3. La **correspondencia de Curry-Howard** para procesos concurrentes:
 - proposiciones en LL $\iff$ session types
 - pruebas en LL $\iff$ procesos concurrentes
 - simplificación de pruebas $\iff$ sincronización entre procesos
4. La correspondencia usando la **versión clásica** de LL.
5. Comunicación asíncrona y deadlock freedom.
—————
6. Extensión de la correspondencia con recursos ilimitados: **clientes y servidores**.
7. La lógica de **implicaciones agrupadas** (*bunched implications*).

Aspectos Prácticos

Cada clase estará dividida en tres partes:

- ▶ 1:30 - 2:15: Parte 1
- ▶ 2:15 - 2:20: Pausa
- ▶ 2:20 - 3:00: Parte 2
- ▶ 3:00 - 3:20: Coffee break
- ▶ 3:20 - 4:30: Parte 3

Material asociado al curso:

<https://jperez.nl/teaching/eci26>

Preguntas, sugerencias, consultas:

j.a.perez@rug.nl (subject: [eci26])

Introduction

This Course: Session Types / Propositions as Sessions

An introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

This Course: Session Types / Propositions as Sessions

An introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- ▶ In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.
We often talk about **session protocols**.

This Course: Session Types / Propositions as Sessions

An introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- ▶ In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.
We often talk about **session protocols**.
- ▶ A disciplined usage of sessions is key to ensure program correctness.
Not only: sessions have elegant **logical foundations**!

This Course: Session Types / Propositions as Sessions

An introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- ▶ In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.
We often talk about **session protocols**.
- ▶ A disciplined usage of sessions is key to ensure program correctness.
Not only: sessions have elegant **logical foundations**!
- ▶ **Linear Logic (LL)**: A resource-oriented view on propositions / protocols.
Linear resources can be used exactly once.

In the rest of the lecture: the language of session types, and basics of linear logic.

Outline

Motivation

- Program Correctness

- Example: Two-Buyer Protocol

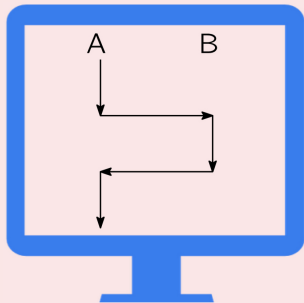
Session Types

- Syntax

- Example: Two-Buyer Protocol

When is a Program Correct?

Sequential Programs



“Programs produce outputs that are consistent with their input”

Concurrent Programs?

Message-Passing Concurrent Programs?

- ▶ Software components (**services**)
distributed across networks

Message-Passing Concurrent Programs

- ▶ Software components (**services**) distributed across networks
- ▶ Coordination takes place by sending and receiving messages

Message-Passing Concurrent Programs

- ▶ Software components (**services**) distributed across networks
- ▶ Coordination takes place by sending and receiving messages
- ▶ Compatible message exchanges are crucial for system correctness

Message-Passing Concurrent Programs

- ▶ Software components (**services**) distributed across networks
- ▶ Coordination takes place by sending and receiving messages
- ▶ Compatible message exchanges are crucial for system correctness
- ▶ Even a single faulty exchange can cause system-wide bugs

Message-Passing Concurrent Programs

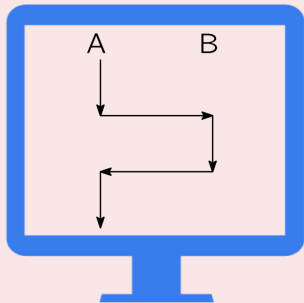
- ▶ Software components (**services**) distributed across networks
- ▶ Coordination takes place by sending and receiving messages
- ▶ Compatible message exchanges are crucial for system correctness
- ▶ Even a single faulty exchange can cause system-wide bugs

An (imperfect) analogy:



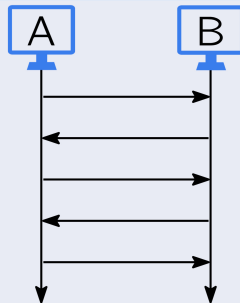
When is a Program Correct?

Sequential Programs



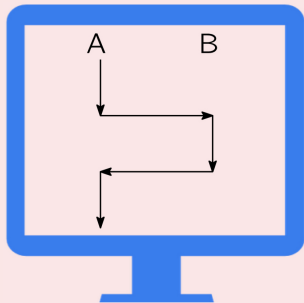
“Programs produce outputs that are consistent with their input”

Concurrent Programs



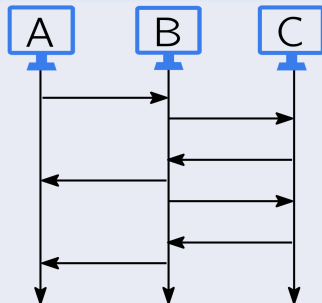
When is a Program Correct?

Sequential Programs



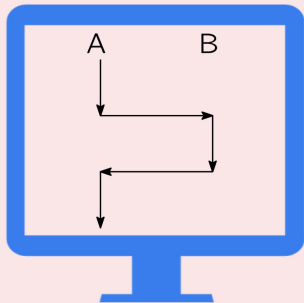
“Programs produce outputs that are consistent with their input”

Concurrent Programs



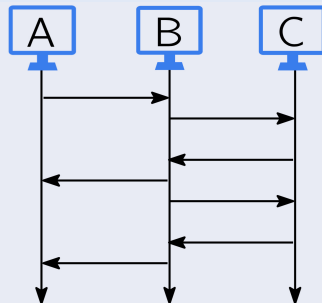
When is a Program Correct?

Sequential Programs



“Programs produce outputs that are consistent with their input”

Concurrent Programs



“Programs always respect their intended **protocols**”

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller**



Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.

Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.

Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction

Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction
4. In case 3(a) Alice contacts Bob to get his address, and forwards it to Seller.

Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction
4. In case 3(a) Alice contacts Bob to get his address, and forwards it to Seller.
- 4'. In case 3(b) Alice is in charge of gracefully concluding the conversation.

Example: A Two-Buyer Protocol



Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction
4. In case 3(a) Alice contacts Bob to get his address, and forwards it to Seller.
- 4'. In case 3(b) Alice is in charge of gracefully concluding the conversation.

Note: The structure of messaging matters!

Example: A Two-Buyer Protocol

Correctness goals for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
 - Alice never ask Bob twice within the same conversation
 - Alice doesn't continue the transaction if Bob can't contribute
 - Alice chooses among the options provided by Seller



Example: A Two-Buyer Protocol

Correctness goals for the implementations of Alice, Bob, and Seller:

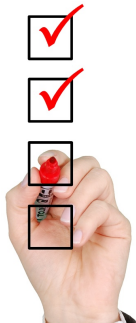
- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
 - Seller always returns an integer when Alice requests a quote



Example: A Two-Buyer Protocol

Correctness goals for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock-Freedom** – they do not **“get stuck”** while running the protocol.
 - Alice eventually receives an answer from Bob on his contribution.



Example: A Two-Buyer Protocol

Correctness goals for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock-Freedom** – they do not “**get stuck**” while running the protocol.
- **Termination** – they do not engage in **infinite behavior** (that may prevent them from completing the protocol)



Example: A Two-Buyer Protocol

Correctness goals for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock-Freedom** – they do not **“get stuck”** while running the protocol.
- **Termination** – they do not engage in **infinite behavior** (that may prevent them from completing the protocol)

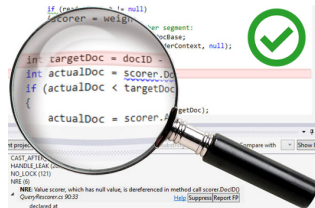


Correctness

- ▶ A **non-trivial notion**, which follows from the interplay of these properties.
- ▶ **Hard to enforce** due to actions being “scattered around” in programs.
- A session specifies a protocol's structure, enabling program verification. It stipulates **what** and **when** should be exchanged (along a channel)

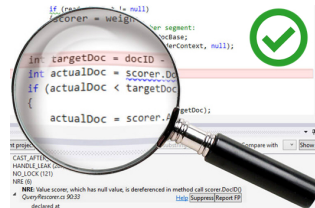
Type Systems

- Can detect bugs before programs are run
 - Attached to many programming languages
 - Implement a specific notion of correctness
- A program is either correct or incorrect



Type Systems

- Can detect bugs before programs are run
 - Attached to many programming languages
 - Implement a specific notion of correctness
- A program is either correct or incorrect

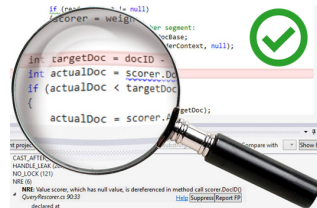


Sequential Languages

- **Data** type systems classify values in a program
- Examples: Integers, strings, etc

Type Systems

- Can detect bugs before programs are run
- Attached to many programming languages
- Implement a specific notion of correctness
A program is either correct or incorrect



Sequential Languages

- **Data** type systems classify values in a program
- Examples: Integers, strings, etc

Concurrent Languages

- **Behavioral** type systems classify protocols in a program
- Example: “first send username, then receive true/false, finally close”
- A typical bug: sending messages in the wrong order

How to Design Type Systems? Logic to the Rescue!



How to Design Type Systems? Logic to the Rescue!

A landmark result in programming language theory:

► **Propositions as types** (Curry, 1935; Howard, 1969)

Propositions in Intuitionistic Logic $\iff$ Types

Proofs in natural deduction $\iff$ Sequential programs (λ -calculus)

Proof simplification $\iff$ Program evaluation (β -reduction)

How to Design Type Systems? Logic to the Rescue!

- ▶ **Propositions as types** (Curry, 1935; Howard, 1969)

Propositions in Intuitionistic Logic $\iff$ Types

Proofs in natural deduction $\iff$ Sequential programs (λ -calculus)

Proof simplification $\iff$ Program evaluation (β -reduction)

- ▶ What about concurrent programs? A **resource-oriented** view!

How to Design Type Systems? Logic to the Rescue!

► Propositions as sessions

Propositions in **Linear Logic** (LL) $\leftrightarrow$ **Session types**

Proofs in LL $\leftrightarrow$ Interacting processes

Cut elimination in LL $\leftrightarrow$ process communication

Outline

Motivation

Program Correctness

Example: Two-Buyer Protocol

Session Types

Syntax

Example: Two-Buyer Protocol

Protocols as Session Types

Session types specify protocols in terms of

- communication actions: send and receive
- choices: offers and selections
- recursion



Protocols as Session Types

Session types specify protocols in terms of

- communication actions: send and receive
- choices: offers and selections
- recursion



Session protocols are attached to **interaction devices**:

- communication channels in programs (think Go and Rust)
- TCP-IP sockets
- ...

Protocols as Session Types

A formal syntax for protocols:

$S ::= !U; S$

send value of type U , continue as S

Protocols as Session Types

A formal syntax for protocols:

$$S ::= !U; S \\ \mid ?U; S$$

send value of type U , continue as S

receive value of type U , continue as S

Protocols as Session Types

A formal syntax for protocols:

$$S ::= \begin{array}{l} !U; S \\ | \quad ?U; S \\ | \quad \&\{l_1 : S_1, \dots, l_n : S_n\} \end{array}$$

send value of type U , continue as S

receive value of type U , continue as S

offer alternatives $S_1, \dots, S_n$

($l_1, \dots, l_n$ are **labels**)

Protocols as Session Types

A formal syntax for protocols:

$S ::= !U; S$

| $?U; S$

| $\&\{l_1 : S_1, \dots, l_n : S_n\}$

| $\oplus\{l_1 : S_1, \dots, l_n : S_n\}$

send value of type U , continue as S

receive value of type U , continue as S

offer alternatives $S_1, \dots, S_n$

($l_1, \dots, l_n$ are **labels**)

select one between $S_1, \dots, S_n$

Protocols as Session Types

A formal syntax for protocols:

$$\begin{array}{l} S ::= !U; S \\ \quad | \quad ?U; S \\ \quad | \quad \&\{l_1 : S_1, \dots, l_n : S_n\} \\ \quad | \quad \oplus\{l_1 : S_1, \dots, l_n : S_n\} \\ \quad | \quad \mu t. S \quad | \quad t \end{array}$$

send value of type U , continue as S

receive value of type U , continue as S

offer alternatives $S_1, \dots, S_n$

($l_1, \dots, l_n$ are **labels**)

select one between $S_1, \dots, S_n$

recursion

Protocols as Session Types

A formal syntax for protocols:

$$\begin{array}{l} S ::= !U; S \\ \quad | \quad ?U; S \\ \quad | \quad \&\{l_1 : S_1, \dots, l_n : S_n\} \\ \quad | \quad \oplus\{l_1 : S_1, \dots, l_n : S_n\} \\ \quad | \quad \mu t. S \quad | \quad t \\ \quad | \quad \text{end} \end{array}$$

send value of type U , continue as S

receive value of type U , continue as S

offer alternatives $S_1, \dots, S_n$

($l_1, \dots, l_n$ are **labels**)

select one between $S_1, \dots, S_n$

recursion

terminated protocol

Protocols as Session Types

A formal syntax for protocols:

$S ::=$	$!U; S$	send value of type U , continue as S
$ $	$?U; S$	receive value of type U , continue as S
$ $	$\&\{l_1 : S_1, \dots, l_n : S_n\}$	offer alternatives $S_1, \dots, S_n$ ($l_1, \dots, l_n$ are labels)
$ $	$\oplus\{l_1 : S_1, \dots, l_n : S_n\}$	select one between $S_1, \dots, S_n$
$ $	$\mu t. S \quad \quad t$	recursion
$ $	end	terminated protocol

Notice:

- Sequential communication patterns (no built-in concurrency)
- U stands for basic values (e.g. `int`) but also other sessions S

Exercises

What do these protocols do?

- ▶ $S_1 \triangleq ?\text{int}; \text{end}$
- ▶ $S_2 \triangleq ?\text{int}; ?\text{int}; !\text{bool}; \text{end}$
- ▶ $S_3 \triangleq !\text{int}; ?\text{bool}; \text{end}$
- ▶ $S_4 \triangleq \&\{l_1 : ?\text{int}; ?\text{int}; !\text{bool}; \text{end}, \quad l_2 : ?\text{str}; !\text{int}; \text{end}\}$
- ▶ $S_5 \triangleq \mu t. ?\text{int}; t$
- ▶ $S_6 \triangleq \mu t. ?\text{int}; ?\text{int}; !\text{bool}; t$
- ▶ $S_7 \triangleq \mu t. \&\{\text{recv} : ?\text{int}; ?\text{int}; !\text{bool}; t, \quad \text{stop} : !\text{str}; \text{end}\}$
- ▶ $S_8 \triangleq \oplus\{l_1 : !\text{int}; !\text{int}; ?\text{bool}; \text{end}, \quad l_2 : !\text{str}; ?\text{int}; \text{end}\}$

The Two-Buyer Protocol, Revisited



Recall the protocol between Alice, Bob, and Seller:

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute to buy the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction
4. In case 3(a) Alice contacts Bob to get his address, and forwards it to Seller.
- 4'. In case 3(b) Alice is in charge of gracefully concluding the conversation.

Session Types for The Two-Buyer Protocol

Two independent protocols, with Alice “leading” the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{SA} = ?\text{book}; !\text{quote}; \& \begin{cases} \text{buy} : & ?\text{paym}; ?\text{address}; !\text{ok}; \text{end} \\ \text{cancel} : & ?\text{thanks}; !\text{bye}; \text{end} \end{cases}$$



Session Types for The Two-Buyer Protocol

Two independent protocols, with Alice “leading” the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{SA} = ?\text{book}; !\text{quote}; \& \begin{cases} \text{buy} : & ?\text{paym}; ?\text{address}; !\text{ok}; \text{end} \\ \text{cancel} : & ?\text{thanks}; !\text{bye}; \text{end} \end{cases}$$

2. A session type for Alice (in its interaction with Bob):

$$S_{AB} = !\text{cost}; \& \begin{cases} \text{share} : & ?\text{address}; !\text{ok}; \text{end} \\ \text{close} : & !\text{bye}; \text{end} \end{cases}$$



Session Types for The Two-Buyer Protocol

Implementations for Alice, Bob, and Seller should be **compatible**.

Session Types for The Two-Buyer Protocol

Implementations for Alice, Bob, and Seller should be **compatible**.

Duality relates session types with opposite behaviors:

- The dual of sending is receiving (and vice versa)
- Branching is the dual of selection (and vice versa)

The dual of session type S is written $\overline{S}$.

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Alice's viewpoint in her interaction with Bob:

$$S_{AB} = !\text{cost}; \& \begin{cases} \text{share} : \text{?address}; !\text{ok}; \text{end} \\ \text{close} : \text{!bye}; \text{end} \end{cases}$$

- Given this, Bob's implementation should conform to $\overline{S_{AB}}$, the dual of S_{AB} :

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Alice's viewpoint in her interaction with Bob:

$$S_{AB} = !\text{cost}; \& \left\{ \begin{array}{l} \text{share} : \text{?address}; !\text{ok}; \text{end} \\ \text{close} : \text{!bye}; \text{end} \end{array} \right.$$

- Given this, Bob's implementation should conform to $\overline{S_{AB}}$, the dual of S_{AB} :

$$\overline{S_{AB}} = \text{?cost}; \oplus \left\{ \begin{array}{l} \text{share} : \text{!address}; \text{?ok}; \text{end} \\ \text{close} : \text{?bye}; \text{end} \end{array} \right.$$

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Alice's viewpoint in her interaction with Bob:

$$S_{AB} = !\text{cost}; \& \left\{ \begin{array}{l} \text{share} : \text{?address}; !\text{ok}; \text{end} \\ \text{close} : \text{!bye}; \text{end} \end{array} \right.$$

- Given this, Bob's implementation should conform to $\overline{S_{AB}}$, the dual of S_{AB} :

$$\overline{S_{AB}} = \text{?cost}; \oplus \left\{ \begin{array}{l} \text{share} : \text{!address}; \text{?ok}; \text{end} \\ \text{close} : \text{?bye}; \text{end} \end{array} \right.$$

- Also, Alice's implementation should conform to both $\overline{S_{SA}}$ and S_{AB} .

Session Type Duality, Formally

Given a (finite) session type S , its dual type $\overline{S}$ is inductively defined as follows:

$$\overline{!U; S} = ?U; \overline{S}$$

$$\overline{?U; S} = !U; \overline{S}$$

$$\overline{\&\{l_i : S_i\}_{i \in I}} = \oplus\{l_i : \overline{S_i}\}_{i \in I}$$

$$\overline{\oplus\{l_i : S_i\}_{i \in I}} = \&\{l_i : \overline{S_i}\}_{i \in I}$$

$$\overline{\text{end}} = \text{end}$$

Session Type Duality, Formally

Given a (finite) session type S , its dual type $\overline{S}$ is inductively defined as follows:

$$\begin{aligned}\overline{!U; S} &= ?U; \overline{S} \\ \overline{?U; S} &= !U; \overline{S} \\ \overline{\&\{l_i : S_i\}_{i \in I}} &= \oplus\{l_i : \overline{S_i}\}_{i \in I} \\ \overline{\oplus\{l_i : S_i\}_{i \in I}} &= \&\{l_i : \overline{S_i}\}_{i \in I} \\ \overline{\text{end}} &= \text{end}\end{aligned}$$

Notice:

- For recursive types, duality is defined **coinductively**
(the dual of $\mu t.S$ is *not* $\mu t.\overline{S}$)

Many Classes of Session Types

There are many classes/variants of session types:

- We overviewed **binary** sessions, where types describe two-party protocols. (We used two separate types to handle Alice, Bob, and Seller.)
- With **multiparty** sessions, types describe protocols involving more than two participants. There is a single **global type** and multiple **local types**.

Many Classes of Session Types (contd.)

There are many classes/variants of session types:

- We saw a syntax of types that describes **regular behaviors**:
in ' $?U; S$ ' and ' $!U; S$ ' we use ';' to compose an action and a protocol.
- In **context-free** session types, we can freely compose two protocols: ' $S_1; S_2$ '.
This is arguably the most expressive class of protocols (to date).

Many Classes of Session Types (contd.)

There are many classes/variants of session types:

- Session types can be attached to programming languages with **synchronous** and **asynchronous** communication.
 - Synchronous: A message is received as soon as it is sent
 - Asynchronous: Message reception is not immediate

These operational differences are reflected in types.

The real world is asynchronous but synchronous is easier to define.

Many Classes of Session Types (contd.)

There are many classes/variants of session types:

- Session types can be attached to programming languages with **synchronous** and **asynchronous** communication.
 - Synchronous: A message is received as soon as it is sent
 - Asynchronous: Message reception is not immediate

These operational differences are reflected in types.

The real world is asynchronous but synchronous is easier to define.

- Session types are **paradigm-independent**, and can be adapted to functional, object-oriented, and actor-based languages with concurrency.

Taking Stock

Up to here:

- Correctness for communicating programs
- Sessions as protocol specifications
- A formal syntax for session types
- Example: A two-buyer protocol
- Protocol compatibility in terms of duality for session types

Coming next:

- Linear logic, in its intuitionistic variant

Linear Logic

Outline

Propositions as Types: The Standard Story

Linear Logic for Resource Management

Sequent Calculus for ILL

From Intuitionistic to Linear Logic

Examples

Intuitionistic Logic (IL)

The grammar of propositions:

$$A, B ::= X \mid A \rightarrow B \mid A \times B \mid A + B$$

where:

- ▶ X is a propositional **constant**
- ▶ $A \rightarrow B$ denotes **implication**
- ▶ $A \times B$ denotes **conjunction**
- ▶ $A + B$ denotes **disjunction**

(alternative notation for $A \wedge B$)

(alternative notation for $A \vee B$)

Intuitionistic Logic (IL)

The grammar of propositions:

$$A, B ::= X \mid A \rightarrow B \mid A \times B \mid A + B$$

where:

- ▶ X is a propositional **constant**
- ▶ $A \rightarrow B$ denotes **implication**
- ▶ $A \times B$ denotes **conjunction** (alternative notation for $A \wedge B$)
- ▶ $A + B$ denotes **disjunction** (alternative notation for $A \vee B$)

Assumptions and judgments:

- ▶ Γ, Δ denote **assumptions**: sequences of zero or more propositions.
- ▶ A **judgment** $\Gamma \vdash A$ means “from assumptions Γ we can conclude A ”.

Rules for Intuitionistic Logic (IL)

Proving judgments of the form $\Gamma \vdash A$.

- ▶ A single **axiom**, a rule without premises
- ▶ **Structural rules** control the ordering, duplication, and discarding of hypotheses in the assumption Γ .
- ▶ Each logical connective is specified in terms of **two rules**: an introduction rule and an elimination rule (this is called **natural deduction**).
- ▶ Using the rules, we aim to construct **derivations** for judgements.

(Later we will see rules in a **sequent calculus**, rather than in natural deduction.)

Rules for IL

Id

$$\frac{}{A \vdash A}$$

Exchange

$$\frac{\Gamma, \Delta \vdash A}{\Delta, \Gamma \vdash A}$$

Contraction

$$\frac{\Gamma, A, A \vdash B}{\Gamma, A \vdash B}$$

Weakening

$$\frac{\Gamma \vdash B}{\Gamma, A \vdash B}$$

$\rightarrow$ -I

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B}$$

$\rightarrow$ -E

$$\frac{\Gamma \vdash A \rightarrow B \quad \Delta \vdash A}{\Gamma, \Delta \vdash B}$$

$\times$ -I

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma, \Delta \vdash A \times B}$$

$\times$ -E

$$\frac{\Gamma \vdash A \times B \quad \Delta, A, B \vdash C}{\Gamma, \Delta \vdash C}$$

$+$ -I₁

$$\frac{\Gamma \vdash A}{\Gamma \vdash A + B}$$

$+$ -I₂

$$\frac{\Gamma \vdash B}{\Gamma \vdash A + B}$$

$+$ -E

$$\frac{\Gamma \vdash A + B \quad \Delta, A \vdash C \quad \Delta, B \vdash C}{\Gamma, \Delta \vdash C}$$

Example of a Derivation

A derivation for the judgment

$$A \rightarrow B, A \vdash A \times B$$

expressed as a **proof tree** (labels for the rules are omitted):

$$\frac{\frac{\frac{A \vdash A}{A \vdash A} \quad \frac{\frac{A \rightarrow B \vdash A \rightarrow B}{A \rightarrow B, A \vdash B}}{A \rightarrow B, A \vdash A \times B}}{A \rightarrow B, A, A \vdash A \times B}}{A \rightarrow B, A \vdash A \times B}$$

We see how A is used twice, with the help of contraction and exchange rules.

Alternative Rules

We have just seen:

$$\frac{\begin{array}{c} \times\text{-I} \\ \Gamma \vdash A \quad \Delta \vdash B \end{array}}{\Gamma, \Delta \vdash A \times B}$$

$$\frac{\begin{array}{c} \times\text{-E} \\ \Gamma \vdash A \times B \quad \Delta, A, B \vdash C \end{array}}{\Gamma, \Delta \vdash C}$$

We can also have:

$$\frac{\begin{array}{c} \times\text{-I}' \\ \Gamma \vdash A \quad \Gamma \vdash B \end{array}}{\Gamma \vdash A \times B}$$

$$\frac{\begin{array}{c} \times\text{-E}_1 \\ \Gamma \vdash A \times B \end{array}}{\Gamma \vdash A}$$

$$\frac{\begin{array}{c} \times\text{-E}_2 \\ \Gamma \vdash A \times B \end{array}}{\Gamma \vdash B}$$

The rules are **equivalent**: they are interderivable using structural rules.

More Alternative Rules

We have just seen:

$$\text{Id} \quad \frac{}{A \vdash A}$$

$$\rightarrow\text{-I} \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B}$$

$$\rightarrow\text{-E} \quad \frac{\Gamma \vdash A \rightarrow B \quad \Delta \vdash A}{\Gamma, \Delta \vdash B}$$

We can also have:

$$\text{Id}' \quad \frac{}{\Gamma, A, \Delta \vdash A}$$

$$\rightarrow\text{-I} \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B}$$

$$\rightarrow\text{-E}' \quad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B}$$

Here again, structural rules ensure that either group of rules can be used.
We prefer to have an explicit treatment of structural rules.

Commuting Conversions

The order in which certain (structural) rules are applied is irrelevant.

An equivalence relation on proofs, denoted $\Longleftrightarrow$, via **commuting conversions**.

Here are the commuting conversions involving contraction and $\rightarrow$ -E:

$$\begin{array}{ccc} \frac{}{\Gamma, C, \Delta \vdash B} & \Longleftrightarrow & \frac{}{\Gamma, C, \Delta \vdash B} \\[2em] \frac{}{\Gamma, \Delta, C \vdash B} & \Longleftrightarrow & \frac{}{\Gamma, \Delta, C \vdash B} \end{array}$$

Commuting Conversions

The order in which certain (structural) rules are applied is irrelevant.

An equivalence relation on proofs, denoted $\Longleftrightarrow$, via **commuting conversions**.

Here are the commuting conversions involving contraction and $\rightarrow$ -E:

$$\frac{\frac{\Gamma, C, C \vdash A \rightarrow B}{\Gamma, C \vdash A \rightarrow B} \quad \Delta \vdash A}{\Gamma, C, \Delta \vdash B} \Longleftrightarrow \frac{\frac{\Gamma, C, C \vdash A \rightarrow B \quad \Delta \vdash A}{\Gamma, C, C, \Delta \vdash B}}{\Gamma, C, \Delta \vdash B}$$
$$\frac{}{\Gamma, \Delta, C \vdash B} \Longleftrightarrow \frac{}{\Gamma, \Delta, C \vdash B}$$

Commuting Conversions

The order in which certain (structural) rules are applied is irrelevant.

An equivalence relation on proofs, denoted $\Longleftrightarrow$, via **commuting conversions**.

Here are the commuting conversions involving contraction and $\rightarrow$ -E:

$$\frac{\frac{\Gamma, C, C \vdash A \rightarrow B}{\Gamma, C \vdash A \rightarrow B} \quad \Delta \vdash A}{\Gamma, C, \Delta \vdash B} \Longleftrightarrow \frac{\frac{\Gamma, C, C \vdash A \rightarrow B \quad \Delta \vdash A}{\Gamma, C, C, \Delta \vdash B}}{\Gamma, C, \Delta \vdash B}$$
$$\frac{\Gamma \vdash A \rightarrow B \quad \frac{\Delta, C, C \vdash A}{\Delta, C \vdash A}}{\Gamma, \Delta, C \vdash B} \Longleftrightarrow \frac{\frac{\Gamma \vdash A \rightarrow B \quad \Delta, C, C \vdash A}{\Gamma, \Delta, C, C \vdash B}}{\Gamma, \Delta, C \vdash B}$$

Proof Reduction

- ▶ A judgement may have several, very different derivations.
- ▶ In particular, it is not very useful to have an introduction rule for a connective that is followed by the corresponding elimination rule.
- ▶ The interest is in **reducing** proofs to avoid unnecessary occurrence of connectives; the proofs themselves may not be “smaller”.
- ▶ Notions of **normal form** and **subformula property** apply for proofs and their reductions.

Proof Reduction: Example

Consider the proof

$$\begin{array}{c}
 \frac{\frac{\frac{\overline{B \vdash B} \quad \overline{B \vdash B}}{B, B \vdash B \times B}}{B \vdash B \times B}}{\vdash B \rightarrow (B \times B)} \quad (\dagger) \quad \frac{\frac{\overline{A \rightarrow B \vdash A \rightarrow B} \quad \overline{A \vdash A}}{A \rightarrow B, A \vdash B}}{A \rightarrow B, A \vdash B \times B} \\
 \hline
 A \rightarrow B, A \vdash B \times B
 \end{array}$$

It can be reduced to:

$$\begin{array}{c}
 \frac{\frac{\overline{A \rightarrow B \vdash A \rightarrow B} \quad \overline{A \vdash A}}{A \rightarrow B, A \vdash B} \quad \frac{\overline{A \rightarrow B \vdash A \rightarrow B} \quad \overline{A \vdash A}}{A \rightarrow B, A \vdash B}}{\frac{A \rightarrow B, A, A \rightarrow B, A \vdash B \times B}{A \rightarrow B, A \vdash B \times B}} \quad (\dagger)
 \end{array}$$

Propositions as Types

Extending judgements to contain terms (= programs!).

- ▶ From the logic perspective, terms encode proofs.
- ▶ From the programmer's perspective, terms are a programming language for which logic provides a **type system**.

Propositions as Types

Propositions can be read as types, and proofs are read as corresponding typed terms:

- ▶ A proof of $A \rightarrow B$ is a **function** that takes any proof of A into a proof of B .
- ▶ A proof of $A \times B$ is a **pair** of a proof of A and a proof of B .
- ▶ A proof of $A + B$ is a **disjoint sum**: either a proof of A or a proof of B .

Terms encode the rules of the logic:

- ▶ An introduction rule: a term that **constructs** a value.
- ▶ An elimination rule: a term that **deconstructs** a value.
- ▶ There are no term forms for the structural rules.

We briefly have a look at the terms (λ -calculus!) and their reductions.

Intuitionistic Terms

$$\begin{aligned} s, t, u, v, w \quad ::= \quad & x \\ & | \lambda x. u \mid s(t) \\ & | (t, u) \mid \text{case } s \text{ of } (x, y) \rightarrow v \\ & | \text{inl}(t) \mid \text{inr}(u) \mid \text{case } s \text{ of } \text{inl}(x) \rightarrow v; \text{inr}(y) \rightarrow w \end{aligned}$$

Assumptions Γ, Δ are now written in the form

$$x_1 : A_1, \dots, x_n : A_n$$

where $n \geq 0$ and

- ▶ $x_1, \dots, x_n$ are distinct variables;
- ▶ $A_1, \dots, A_n$ are types (= propositions).

Judgments are then of the form $\Gamma \vdash t : A$ (“term t has type A ”).

Types for Intuitionistic Terms

Id $\frac{}{x:A \vdash x:A}$	Exchange $\frac{\Gamma, \Delta \vdash A}{\Delta, \Gamma \vdash A}$	Contraction $\frac{\Gamma, y:A, z:A \vdash u:B}{\Gamma, x:A \vdash u[x/y, x/z]:B}$	Weakening $\frac{\Gamma \vdash u:B}{\Gamma, x:A \vdash u:B}$
$\frac{\rightarrow\text{-I} \quad \Gamma, x:A \vdash u:B}{\Gamma \vdash \lambda x. u: A \rightarrow B}$	$\frac{\rightarrow\text{-E} \quad \Gamma \vdash s: A \rightarrow B \quad \Delta \vdash t:A}{\Gamma, \Delta \vdash s(t): B}$		
$\frac{\times\text{-I} \quad \Gamma \vdash t:A \quad \Delta \vdash u:B}{\Gamma, \Delta \vdash (t, u): A \times B}$	$\frac{\times\text{-E} \quad \Gamma \vdash s: A \times B \quad \Delta, x:A, y:B \vdash v:C}{\Gamma, \Delta \vdash \text{case } s \text{ of } (x, y) \rightarrow v: C}$		

Types for Intuitionistic Terms

$$\frac{+-l_1 \quad \Gamma \vdash t : A}{\Gamma \vdash \text{inl}(t) : A + B}$$

$$\frac{+-l_2 \quad \Gamma \vdash u : B}{\Gamma \vdash \text{inr}(u) : A + B}$$

$$\frac{+-E \quad \Gamma \vdash s : A + B \quad \Delta, x : A \vdash v : C \quad \Delta, y : B \vdash w : C}{\Gamma, \Delta \vdash \text{case } s \text{ of } \text{inl}(x) \rightarrow v; \text{inr}(y) \rightarrow w : C}$$

Typing Derivations: Example

Recall the derivation for the judgment $A \rightarrow B, A \vdash A \times B$, seen before.
Now with terms:

$$\frac{\frac{\frac{\overline{y:A \vdash y:A}}{y:A, f:A \rightarrow B, z:A \vdash (y, f(z)):A \times B} \quad \frac{\frac{\overline{f:A \rightarrow B \vdash f:A \rightarrow B} \quad \overline{z:A \vdash z:A}}{f:A \rightarrow B, z:A \vdash f(z):B}}{f:A \rightarrow B, y:A, z:A \vdash (y, f(z)):A \times B}}{f:A \rightarrow B, x:A \vdash (x, f(x)):A \times B}$$

Term Reduction

Since terms encode proofs, proof reductions in logic correspond to term reductions:

$$\lambda x. u (t) \iff u[t/x]$$

$$\text{case } (t, u) \text{ of } (x, y) \rightarrow v \iff u[t/x, v/y]$$

$$\text{case inl } (t) \text{ of inl } (x) \rightarrow v; \text{ inr } (y) \rightarrow w \iff v[t/x]$$

$$\text{case inr } (u) \text{ of inl } (x) \rightarrow v; \text{ inr } (y) \rightarrow w \iff w[u/x]$$

Notice:

The notion of proof reduction, associated to properties of proof systems, corresponds precisely to term reduction for a model of computation!

Outline

Propositions as Types: The Standard Story

Linear Logic for Resource Management

Sequent Calculus for ILL

From Intuitionistic to Linear Logic

Examples

Linear Logic: A Substructural Logic for Computation

- ▶ Introduced by Jean-Yves Girard in 1987.
- ▶ Observation: Contraction arbitrarily duplicates assumptions; weakening freely discards them. Not always sensible for computational resources!
- ▶ Linear logic advocates a **disciplined** use of resources, by controlling contraction and weakening. Many applications in proof theory and computer science.
- ▶ A “resource-aware” logic:
 - ▶ $A \vdash B$: given A -resources, a way to obtain B -resources
 - ▶ Proof theoretically: **substructural logic**

Linear Logic

- ▶ We are mostly interested in the intuitionistic variant, dubbed ILL. We will also have a look at classical LL (CLL).
- ▶ We see rules as a sequent calculus, rather than as natural deduction.
 - ▶ In ND: introduction and elimination rules
 - ▶ In sequent calculi: left and right rules, plus the cut rule

Linear Logic: Propositions

' A is true' vs 'I have A -resources'

$$A \rightarrow B$$

if A is true, then B is true

$$A \wedge B$$

both A and B are true

Linear Logic: Propositions

' A is true' vs 'I have A -resources'

$$A \rightarrow B$$

if A is true, then B is true

$$A \wedge B$$

both A and B are true

$$A \multimap B \quad \text{if you give me } A, \text{ then I can produce } B$$

Linear Logic: Propositions

' A is true' vs 'I have A -resources'

$$A \rightarrow B$$

if A is true, then B is true

$$A \wedge B$$

both A and B are true

$$A \multimap B \quad \text{if you give me } A, \text{ then I can produce } B$$

$$A \otimes B \quad \text{I have both } A \text{ and } B$$

Linear Logic: Propositions

' A is true' vs 'I have A -resources'

$$A \rightarrow B$$

if A is true, then B is true

$$A \wedge B$$

both A and B are true

$$A \multimap B \quad \text{if you give me } A, \text{ then I can produce } B$$

$$A \otimes B \quad \text{I have both } A \text{ and } B$$

$$A \& B \quad \text{Choose from } A \text{ and } B$$

Examples:

▶ $euro \otimes euro \multimap pizza$

▶ $dough \otimes (sauce \otimes toppings) \multimap pizza$

Linear Logic: Linearity

$$A \rightarrow A \wedge A$$

$$A \not\vdash_{\circ} A \otimes A$$

$$A \wedge B \rightarrow A$$

$$A \otimes B \not\vdash_{\circ} A$$

Linear Logic: Linearity

$$A \rightarrow A \wedge A$$

$$A \not\vdash_{\circ} A \otimes A$$

$$A \wedge B \rightarrow A$$

$$A \otimes B \not\vdash_{\circ} A$$

$$A \wedge (A \rightarrow B) \rightarrow B$$

$$A \otimes (A \multimap B) \multimap B$$

Linear Logic: Linearity

$$A \rightarrow A \wedge A$$

$$A \not\multimap A \otimes A$$

$$A \wedge B \rightarrow A$$

$$A \otimes B \not\multimap A$$

$$A \wedge (A \rightarrow B) \rightarrow B$$

$$A \otimes (A \multimap B) \multimap B$$

$$A \wedge (A \rightarrow B) \rightarrow A \wedge B$$

$$A \otimes (A \multimap B) \not\multimap A \otimes B$$

Linear Logic is **substructural**, resource-aware.

Outline

Propositions as Types: The Standard Story

Linear Logic for Resource Management

Sequent Calculus for ILL

From Intuitionistic to Linear Logic

Examples

Proof Theory of $\wedge$

Sequent calculus: $\Gamma \vdash A$

$$\frac{}{A \vdash A}$$

$$\frac{\wedge R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}$$

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

Proof Theory of $\wedge$

Assertion A holds under the list of hypotheses Γ .

Intuitively, $\wedge \Gamma \Rightarrow A$

Sequent calculus: $\Gamma \vdash A$

$$\frac{}{A \vdash A}$$

$$\frac{\wedge R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}$$

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$

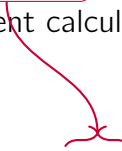
$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

Proof Theory of $\wedge$

Axiom rule

Sequent calculus: $\Gamma \vdash A$


$$\frac{}{A \vdash A}$$

$$\frac{\wedge R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}$$

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

Proof Theory of $\wedge$

Sequent calculus: $\Gamma \vdash A$

Right and left rules for $\wedge$

$$\begin{array}{c} \frac{}{A \vdash A} \\[1em] \frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B} \quad \wedge R \\[1em] \frac{\Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C} \quad \wedge L \\[1em] \frac{\Gamma \vdash C}{\Gamma, A \vdash C} \quad \text{Weakn} \\[1em] \frac{\Gamma, A, A \vdash C}{\Gamma, A \vdash C} \quad \text{Contr} \\[1em] \frac{\Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C} \quad \text{eXchg} \end{array}$$

Proof Theory of $\wedge$

Sequent calculus: $\Gamma \vdash A$

$$\begin{array}{c} \frac{}{A \vdash A} \\[1em] \frac{\wedge R}{\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}} \\[1em] \frac{\wedge L}{\frac{\Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}} \\[1em] \frac{\text{Weakn}}{\frac{\Gamma \vdash C}{\Gamma, A \vdash C}} \quad \frac{\text{Contr}}{\frac{\Gamma, A, A \vdash C}{\Gamma, A \vdash C}} \quad \frac{\text{eXchg}}{\frac{\Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}} \end{array}$$

Structural rules

Proof Theory of $\wedge$

$\Gamma : \text{Frml} \rightarrow \mathbb{N}$ is a multiset

Sequent calculus: $\Gamma \vdash A$

$$\frac{}{A \vdash A}$$

$$\frac{\wedge R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}$$

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

We can omit the exchange rule by treating Γ as a multiset

Generalized Structural Rules

$$\frac{\Gamma, \Gamma \vdash C}{\Gamma \vdash C}$$

$$\frac{\Gamma \vdash C}{\Gamma, \Delta \vdash C}$$

Obtained from repeatedly using the usual rules (and exchange).

Using Structural Rules

$$\frac{}{A \wedge B \vdash A}$$

$$\frac{}{A \vdash A \wedge A}$$

Using Structural Rules

$$\frac{A, B \vdash A}{A \wedge B \vdash A}$$

$$\frac{}{A \vdash A \wedge A}$$

Using Structural Rules

$$\frac{A \vdash A}{\frac{A, B \vdash A}{A \wedge B \vdash A}}$$

$$\frac{}{A \vdash A \wedge A}$$

Using Structural Rules

$$\frac{\frac{A \vdash A}{A, B \vdash A}}{A \wedge B \vdash A}$$

$$\frac{}{A, A \vdash A \wedge A}$$
$$\frac{}{A \vdash A \wedge A}$$

Using Structural Rules

$$\frac{A \vdash A}{\frac{A, B \vdash A}{A \wedge B \vdash A}}$$

$$\frac{\frac{A \vdash A \quad A \vdash A}{A, A \vdash A \wedge A}}{A \vdash A \wedge A}$$

Using Structural Rules

$$\frac{A \vdash A}{\frac{A, B \vdash A}{A \wedge B \vdash A}}$$

$$\frac{\frac{A \vdash A \quad A \vdash A}{A, A \vdash A \wedge A}}{A \vdash A \wedge A}$$

The usage of the contraction and weakening rules is crucial!

Associativity

$$\frac{}{A \wedge (B \wedge C) \vdash (A \wedge B) \wedge C}$$

Associativity

$$\frac{A, B, C \vdash (A \wedge B) \wedge C}{A \wedge (B \wedge C) \vdash (A \wedge B) \wedge C}$$

Associativity

$$\frac{\frac{\overline{A, B \vdash A \wedge B} \quad C \vdash C}{A, B, C \vdash (A \wedge B) \wedge C}}{A \wedge (B \wedge C) \vdash (A \wedge B) \wedge C}$$

Associativity

$$\frac{\frac{\frac{A \vdash A \quad B \vdash B}{A, B \vdash A \wedge B} \quad C \vdash C}{A, B, C \vdash (A \wedge B) \wedge C}}{A \wedge (B \wedge C) \vdash (A \wedge B) \wedge C}$$

Alternative Rules for $\wedge$

We can replace $\wedge L$ with the following two rules:

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C} \longrightarrow \frac{\wedge L_1 \quad \Gamma, A \vdash C}{\Gamma, A \wedge B \vdash C} \quad \frac{\wedge L_2 \quad \Gamma, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

These rules **internalize weakening**.

Alternative Rules for $\wedge$

We can replace $\wedge R$ with the following rule:

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B} \longrightarrow \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$$

Outline

Propositions as Types: The Standard Story

Linear Logic for Resource Management

Sequent Calculus for ILL

From Intuitionistic to Linear Logic

Examples

From Intuitionistic to Linear Logic

- ▶ Main idea: treat $\otimes$ the same as $\wedge$, minus contraction and weakening rules.
- ▶ The context Γ then externalizes $\otimes$ on the meta-level:

$$\Gamma \vdash A \quad \longrightarrow \quad \bigotimes \Gamma \Longrightarrow A$$

- ▶ We will recover contraction and weakening later with the **! modality**.

Proof Theory of $\otimes$

Sequent calculus: $\Gamma \vdash A$

$$\frac{}{A \vdash A}$$

$$\frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B}$$

$$\frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}$$

~~$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$~~

~~$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$~~

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

Proof Theory of $\otimes$

Sequent calculus: $\Gamma \vdash A$

Left and right rules for $\otimes$

$$\frac{}{A \vdash A}$$

$$\frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B}$$

$$\frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}$$

~~$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$~~

~~$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$~~

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

Proof Theory of $\otimes$

Sequent calculus: $\Gamma \vdash A$

$$\frac{}{A \vdash A}$$

$$\frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B}$$

$$\frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}$$

~~$$\frac{\text{Weakn} \quad \Gamma \vdash C}{\Gamma, A \vdash C}$$~~

~~$$\frac{\text{Contr} \quad \Gamma, A, A \vdash C}{\Gamma, A \vdash C}$$~~

$$\frac{\text{eXchg} \quad \Gamma, A, B, \Gamma' \vdash C}{\Gamma, B, A, \Gamma' \vdash C}$$

No weakening or contraction

Adding the Unit

A proof theory with just $\otimes$ is not fun.

We need multiplicative unit (1) to signify **absence of resources**:

$$\frac{}{\emptyset \vdash 1}$$

$$\frac{\Gamma \vdash C}{\Gamma, 1 \vdash C}$$

Adding Implication

A proof theory with just $\otimes, 1$ is not fun.

We need linear implication ($\multimap$, *lollipop*) to form linear hypotheticals:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B}$$

$$\frac{}{\Gamma_1, \Gamma_2, A \multimap B \vdash C}$$

Adding Implication

A proof theory with just $\otimes, 1$ is not fun.

We need linear implication ($\multimap$, *lollipop*) to form linear hypotheticals:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2, B \vdash C}{\Gamma_1, \Gamma_2, A \multimap B \vdash C}$$

Exercise: try to derive $(A \otimes B) \multimap C \dashv\vdash A \multimap (B \multimap C)$

Cut rule

$$\text{Cut} \frac{\Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B}$$

What is so special about this rule?

Each cut rule introduces a piece of “complexity” as a new formula A

Cut rule

$$\text{Cut} \frac{\Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B}$$

What is so special about this rule?

Each cut rule introduces a piece of “complexity” as a new formula A

Theorem

Every proof of $\Gamma \vdash A$ that uses cut can be converted into a proof that does not use cut

MILL

$$A, B ::= 1 \mid A \otimes B \mid A \multimap B$$

$$\frac{}{A \vdash A}$$

$$\frac{}{\emptyset \vdash 1}$$

$$\frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B}$$

$$\frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}$$

$$\frac{\text{Cut} \quad \Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B}$$

$$\frac{\multimap R \quad \Gamma, A \vdash B}{\Gamma \vdash A \multimap B}$$

$$\frac{\multimap L \quad \Gamma_1 \vdash A \quad \Gamma_2, B \vdash C}{\Gamma_1, \Gamma_2, A \multimap B \vdash C}$$

Adding &

Recall the different versions of left/right rules for $\wedge$

$$\frac{\wedge R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \wedge B}$$

$$\frac{\wedge R' \quad \Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$$

$$\frac{\wedge L \quad \Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}$$

$$\frac{\wedge L_i \quad \Gamma, A_i \vdash C}{\Gamma, A_1 \wedge A_2 \vdash C}$$

Adding &

Recall the different versions of left/right rules for $\wedge$

$$\begin{array}{c} \otimes R \\ \frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B} \end{array}$$

$$\begin{array}{c} \wedge R' \\ \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \end{array}$$

$$\begin{array}{c} \otimes L \\ \frac{\Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C} \end{array}$$

$$\begin{array}{c} \wedge L_i \\ \frac{\Gamma, A_i \vdash C}{\Gamma, A_1 \wedge A_2 \vdash C} \end{array}$$

Adding &

Recall the different versions of left/right rules for $\wedge$

$$\begin{array}{c} \otimes R \\ \frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B} \end{array}$$

$$\begin{array}{c} \& R \\ \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \end{array}$$

$$\begin{array}{c} \otimes L \\ \frac{\Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C} \end{array}$$

$$\begin{array}{c} \& L_i \\ \frac{\Gamma, A_i \vdash C}{\Gamma, A_1 \& A_2 \vdash C} \end{array}$$

Interplay of $\otimes$ and $\&$

- ▶ $A \& B \vdash A$, and $A \& B \vdash B$
- ▶ $A \& B \not\vdash A \otimes B$
- ▶ $(A \multimap B) \& (A \multimap C) \vdash A \multimap B \& C$
- ▶ $(A \& B) \multimap C \not\vdash A \multimap (B \multimap C)$

Outline

Propositions as Types: The Standard Story

Linear Logic for Resource Management

Sequent Calculus for ILL

From Intuitionistic to Linear Logic

Examples

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$E^{15} \multimap$



Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$$E^{15} \multimap_o \left((Sal \ \& \ Soup) \otimes \right)$$

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$$E^{15} \multimap \left((Sal \& Soup) \otimes \left((Schn \& Tofu \& (E \otimes E \multimap Fish)) \right) \right)$$

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \ \& \ Soup) \otimes \\ ((Schn \ \& \ Tofu \ \& \ (E \otimes E \multimap Fish)) \\ \otimes \ Fries) \otimes \end{array} \right)$$

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \& Soup) \otimes \\ ((Schn \& Tofu \& (E \otimes E \multimap Fish)) \\ \otimes Fries) \otimes \\ (Icecr \& Cheese) \otimes \end{array} \right)$$

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (for
additional €3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \& Soup) \otimes \\ ((Schn \& Tofu \& (E \otimes E \multimap Fish)) \\ \otimes Fries) \otimes \\ (Icecr \& Cheese) \otimes \\ (E^3 \multimap Beer) \end{array} \right)$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\right) \multimap \textit{Pizza}$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\text{Yeast} \otimes \text{Flour} \otimes \right) \\ \multimap \text{Pizza}$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\begin{array}{l} \textit{Yeast} \otimes \textit{Flour} \otimes \\ \textit{Salt} \otimes \textit{Sugar} \otimes \end{array} \right) \multimap \textit{Pizza}$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\begin{array}{l} \text{Yeast} \otimes \text{Flour} \otimes \\ \text{Salt} \otimes \text{Sugar} \otimes \\ \text{Tomatoes} \otimes \end{array} \right) \multimap \text{Pizza}$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\begin{array}{l} \textit{Yeast} \otimes \textit{Flour} \otimes \\ \textit{Salt} \otimes \textit{Sugar} \otimes \\ \textit{Tomatoes} \otimes \\ (\textit{Sausage} \text{ or } \textit{Paprika}) \end{array} \right) \multimap \textit{Pizza}$$

Grandma's Linear Logic Pizza recipe

LL Pizza Ingredients:

- ▶ Yeast and flour,
- ▶ Salt and sugar
- ▶ Tomatoes
- ▶ Sausage or paprika

$$\left(\begin{array}{l} \text{Yeast} \otimes \text{Flour} \otimes \\ \text{Salt} \otimes \text{Sugar} \otimes \\ \text{Tomatoes} \otimes \\ (\text{Sausage} \oplus \text{Paprika}) \end{array} \right) \multimap \text{Pizza}$$

Adding $\oplus$

$$\frac{\oplus R_1 \quad \Gamma \vdash A_1}{\Gamma \vdash A_1 \oplus A_2}$$

$$\frac{\oplus R_2 \quad \Gamma \vdash A_2}{\Gamma \vdash A_1 \oplus A_2}$$

$$\frac{\oplus L \quad \Gamma, A_1 \vdash C \quad \Gamma, A_2 \vdash C}{\Gamma, A_1 \oplus A_2 \vdash C}$$

$\oplus$ vs $\&$

The rules are the same as for $\vee$ in the intuitionistic sequent calculus.

$\oplus$ vs $\&$

The rules are the same as for $\vee$ in the intuitionistic sequent calculus.

However,

$$E \& (B \oplus C) \not\vdash (E \& B) \oplus (E \& C)$$

- ▶ E = one euro;
- ▶ B = tea, C = coffee;
- ▶ $(B \oplus C)$ = either tea or coffee, but do not know which;
- ▶ $E \& (B \oplus C)$ = I can either have an euro or a beverage

$\oplus$ vs $\&$

$tea \oplus coffee \vdash awake$

$tea \& coffee \vdash awake$

$\oplus$ vs $\&$

$$tea \oplus coffee \vdash awake$$

Given either tea or coffee (I don't care which one), I can drink it and get awake

$$tea \& coffee \vdash awake$$

$\oplus$ vs $\&$

$tea \oplus coffee \vdash awake$

Given either tea or coffee (I don't care which one), I can drink it and get awake

$tea \& coffee \vdash awake$

Given a choice of tea and coffee (for example at a hotel breakfast), I can drink a hot beverage and get awake

$\oplus$ vs $\&$

$$tea \oplus coffee \vdash awake$$

Given either tea or coffee (I don't care which one), I can drink it and get awake

$$tea \& coffee \vdash awake$$

Given a choice of tea and coffee (for example at a hotel breakfast), I can drink a hot beverage and get awake

$$tea \& coffee \vdash tea \oplus coffee$$

$\oplus$ vs $\&$

$tea \oplus coffee \vdash awake$

Given either tea or coffee (I don't care which one), I can drink it and get awake

$tea \& coffee \vdash awake$

Given a choice of tea and coffee (for example at a hotel breakfast), I can drink a hot beverage and get awake

$tea \& coffee \vdash tea \oplus coffee$

$A \& B \vdash A \oplus B$

$A \oplus B \not\vdash A \& B$

$A, B ::= 1 \mid A \otimes B \mid A \multimap B \mid A \& B \mid A \oplus B$

$$\frac{}{A \vdash A} \quad \frac{}{\emptyset \vdash 1} \quad \frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B} \quad \frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}$$

$$\frac{\text{Cut} \quad \Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B} \quad \frac{\multimap R \quad \Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \quad \frac{\multimap L \quad \Gamma_1 \vdash A \quad \Gamma_2, B \vdash C}{\Gamma_1, \Gamma_2, A \multimap B \vdash C} \quad \frac{\& R \quad \Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B}$$

$$\frac{\& L_i \quad \Gamma, A_i \vdash C}{\Gamma, A_1 \& A_2 \vdash C} \quad \frac{\oplus R_i \quad \Gamma \vdash A_i}{\Gamma \vdash A_1 \oplus A_2} \quad \frac{\oplus L \quad \Gamma, A_1 \vdash C \quad \Gamma, A_2 \vdash C}{\Gamma, A_1 \oplus A_2 \vdash C}$$

Related Logics

- ▶ Classical Linear Logic

Related Logics

- ▶ Classical Linear Logic
- ▶ Relevance logics
 - ▶ Contraction but no weakening
 - ▶ Intuition: all hypothesis are relevant and must be used

Related Logics

- ▶ Classical Linear Logic
- ▶ Relevance logics
 - ▶ Contraction but no weakening
 - ▶ Intuition: all hypothesis are relevant and must be used
- ▶ Logic of Bunched Implications
 - ▶ Better known as the kernel of Separation Logic
 - ▶ Popular in deductive program verification

Taking Stock

Up to here:

- ▶ Correctness for communicating programs
- ▶ Sessions as protocol specifications
- ▶ A formal syntax for session types
- ▶ Example: A two-buyer protocol
- ▶ Protocol compatibility in terms of duality for session types
- ▶ **Intuitionistic Linear Logic (MAILL)**

Tomorrow:

- ▶ Interpreting ILL as session types for message-passing processes

Cut Elimination as Communication

This Course: Propositions as Sessions

We shall explore the logical foundations of concurrent computation.

Plan:

1. Motivation (Jorge) - Multiplicative, Additive Linear Logic (MAILL) (Dan)

This Course: Propositions as Sessions

We shall explore the logical foundations of concurrent computation.

Plan:

1. Motivation (Jorge) - Multiplicative, Additive Linear Logic (MAILL) (Dan)
2. The concurrent interpretation of MAILL (Jorge)
3. **Today:** Cut-elimination and correctness for concurrent processes (Jorge)
4. Beyond linear resources: the $!$ -modality and resource sharing (Dan)
5. An alternative view of resource sharing: Bunched Implications (Dan)

Your questions and feedback are warmly welcome!

Outline

Preliminaries

Computational Interpretation of LL: Statics

- Sequent Calculus

- Output and Input

- Unit and Axiom

- Additives

- Cut

- Processes

Dynamics

- Cut Reduction

- Process Reduction

- Properties

The Two-Buyer Protocol (TBR)



Recall the protocol between Alice, Bob, and Seller:

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
 - a) completing the payment and arranging delivery details
 - b) canceling the transaction
4. In case 3(a) Alice contacts Bob to get his address, and forwards it to Seller.
- 4'. In case 3(b) Alice is in charge of gracefully concluding the conversation.

Session Types for The Two-Buyer Protocol (TBR)

Two independent protocols, with Alice “leading” the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{SA} = ?\text{book}; !\text{quote}; \& \begin{cases} \text{buy} : & ?\text{paym}; ?\text{address}; !\text{ok}; \text{end} \\ \text{cancel} : & ?\text{thanks}; !\text{bye}; \text{end} \end{cases}$$

2. A session type for Alice (in its interaction with Bob):

$$S_{AB} = !\text{cost}; \& \begin{cases} \text{share} : & ?\text{address}; !\text{ok}; \text{end} \\ \text{close} : & !\text{bye}; \text{end} \end{cases}$$



Session Types for The Two-Buyer Protocol (TBR)

Two independent protocols, with Alice “leading” the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{SA} = ?\text{book}; !\text{quote}; \& \begin{cases} \text{buy} : & ?\text{paym}; ?\text{address}; !\text{ok}; \text{end} \\ \text{cancel} : & ?\text{thanks}; !\text{bye}; \text{end} \end{cases}$$

2. A session type for Alice (in its interaction with Bob):

$$S_{AB} = !\text{cost}; \& \begin{cases} \text{share} : & ?\text{address}; !\text{ok}; \text{end} \\ \text{close} : & !\text{bye}; \text{end} \end{cases}$$



Example: A Two-Buyer Protocol

Correctness follows from the interplay of the following properties:

- **Fidelity** – implementations **follow the intended protocol**.
 - Alice never ask Bob twice within the same conversation
 - Alice doesn't continue the transaction if Bob can't contribute
 - Alice chooses among the options provided by Seller



Example: A Two-Buyer Protocol

Correctness follows from the interplay of the following properties:

- **Fidelity** – implementations **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
 - Seller always returns an integer when Alice requests a quote



Example: A Two-Buyer Protocol

Correctness follows from the interplay of the following properties:

- **Fidelity** – implementations **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock-Freedom** – they do not **“get stuck”** while running the protocol.
 - Alice eventually receives an answer from Bob on his contribution.



Example: A Two-Buyer Protocol

Correctness follows from the interplay of the following properties:

- **Fidelity** – implementations **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock-Freedom** – they do not “**get stuck**” while running the protocol.
- **Termination** – they do not engage in **infinite behavior** (that may prevent them from completing the protocol)



MAIL

$$A, B ::= 1 \mid A \otimes B \mid A \multimap B \mid A \& B \mid A \oplus B$$

$$\begin{array}{c} A \vdash A \qquad \emptyset \vdash 1 \qquad \frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B} \qquad \frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C} \\[2ex] \frac{\text{Cut} \quad \Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B} \qquad \frac{\multimap R \quad \Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \qquad \frac{\multimap L \quad \Gamma_1 \vdash A \quad \Gamma_2, B \vdash C}{\Gamma_1, \Gamma_2, A \multimap B \vdash C} \\[2ex] \frac{\& R \quad \Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \qquad \frac{\& L_i \quad \Gamma, A_i \vdash C}{\Gamma, A_1 \& A_2 \vdash C} \qquad \frac{\oplus R_i \quad \Gamma \vdash A_i}{\Gamma \vdash A_1 \oplus A_2} \qquad \frac{\oplus L \quad \Gamma, A_1 \vdash C \quad \Gamma, A_2 \vdash C}{\Gamma, A_1 \oplus A_2 \vdash C} \end{array}$$

Outline

Preliminaries

Computational Interpretation of LL: Statics

- Sequent Calculus

- Output and Input

- Unit and Axiom

- Additives

- Cut

- Processes

Dynamics

- Cut Reduction

- Process Reduction

- Properties

Propositions As Types

The sequential case (aka Curry-Howard correspondence, formulae-as-types, proofs-as-programs...):

Intuitionistic logic propositions	$\leftrightarrow$	types describing data
Natural deduction derivations	$\leftrightarrow$	terms in the λ -calculus
Proof normalization reductions	$\leftrightarrow$	β -reductions

Propositions As Sessions

Today, the concurrent case:

Linear logic propositions	$\leftrightarrow$	types describing behavior (sessions)
Sequent calculus derivations	$\leftrightarrow$	processes in the π -calculus
Cut reductions	$\leftrightarrow$	communication between processes

Propositions As Sessions

Today, the concurrent case:

- Linear logic** propositions $\leftrightarrow$ types describing **behavior** (sessions)
- Sequent calculus** derivations $\leftrightarrow$ **processes** in the π -calculus
- Cut reductions** $\leftrightarrow$ communication between processes

We shall follow the correspondence between session types and intuitionistic linear logic (aka π DILL, Caires & Pfenning 2010).

Linear Logic: Sequent Calculus

The sequent

$$A_1, \dots, A_n \vdash B,$$

is interpreted as $A_1 \otimes \dots \otimes A_n \multimap B$.

Linear Logic: Sequent Calculus

The sequent

$$A_1, \dots, A_n \vdash B,$$

is interpreted as $A_1 \otimes \dots \otimes A_n \multimap B$.

Structural rules:

$$\begin{array}{c} A \vdash A \\[10pt] \frac{\Delta_1 \vdash A \quad \Delta_2, A \vdash B}{\Delta_1, \Delta_2 \vdash B} \quad \frac{\Delta_1, A, B, \Delta_2 \vdash C}{\Delta_1, B, A, \Delta_2 \vdash C} \end{array}$$

Notice: Each connective is “explained” in sequent calculus with a left rule, a right rule, and the interactions with the cut rule.

Linear Logic: Sequent Calculus

$$\frac{\Delta_1 \vdash A \quad \Delta_2 \vdash B}{\Delta_1, \Delta_2 \vdash A \otimes B}$$

$$\frac{\Delta, A, B \vdash C}{\Delta, A \otimes B \vdash C}$$

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta_1 \vdash A \quad B, \Delta_2 \vdash C}{\Delta_1, A \multimap B, \Delta_2 \vdash C}$$

$$\emptyset \vdash 1$$

$$\frac{\Delta \vdash C}{\Delta, 1 \vdash C}$$

Key Ideas

- We shall consider a language of **processes** (denoted $P, Q, \dots$) that interact by synchronizing on **names** (denoted $x, y, z, \dots$).

Key Ideas

- We shall consider a language of **processes** (denoted $P, Q, \dots$) that interact by synchronizing on **names** (denoted $x, y, z, \dots$).
- Processes can send/receive messages on names, using sequencing and parallel composition. We shall gradually “extract” their syntax from proofs.

Key Ideas

- Interpret the logical sequent

$$A_1, \dots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: z : C$$



Process P offers session C on channel z ...

Key Ideas

- Interpret the logical sequent

$$A_1, \dots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: z : C$$

Process P **offers** session C on channel z ...

... by **relying on** sessions $A_1, \dots, A_n$ on channels $x_1, \dots, x_n$

Recall: The Process Language

$P, Q ::=$	$[y \leftarrow x]$	forwarder between sessions x and y
	$(\nu y)(\bar{x}\langle y \rangle.(P \mid Q))$	send y over x , then execute P and Q
	$x(y).P$	receive y over x , then execute P
	$\bar{x}\langle \rangle$	close session x
	$x().P$	wait-close session x , then execute P
	$x \triangleleft l_i; P$	select label l_i along x , then execute P
	$x \triangleright \{l_1 : P_1, \dots, l_n : P_n\}$	branch on x , offering labels $l_1, \dots, l_n$
	0	inaction (silent interpretation)
	$(\nu x)(P \mid Q)$	parallel composition of P and Q on x

Interpreting LL Propositions as Session Types

$!U; S$	send value of type U , continue as S	??
$?U; S$	receive value of type U , continue as S	??
end	terminate the session	??

Notice: A non-commutative reading of $\otimes$!

Interpreting LL Propositions as Session Types

$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$??$
end	terminate the session	$??$

Notice: A non-commutative reading of $\otimes$!

Interpreting LL Propositions as Session Types

$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$U \multimap S$
end	terminate the session	$??$

Interpreting LL Propositions as Session Types

$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$U \multimap S$
end	terminate the session	1

Examples (TBR)

Let's consider again our typing judgment by looking at some examples:

$$x_1 : A_1, x_2 : A_2 \vdash P :: y : C \otimes D$$

$$\emptyset \vdash Q :: y : A \multimap B$$

$$x : A \otimes B, y : C \otimes D \vdash R :: z : 1$$

Examples (TBR)

Let's consider again our typing judgment by looking at some examples:

$$x_1 : A_1, x_2 : A_2 \vdash P :: y : C \otimes D$$

P offers an output along y , if its environment provides session behaviors A_1 and A_2 along x_1 and x_2 , respectively.

$$\emptyset \vdash Q :: y : A \multimap B$$

$$x : A \otimes B, y : C \otimes D \vdash R :: z : 1$$

Examples (TBR)

Let's consider again our typing judgment by looking at some examples:

$$x_1 : A_1, x_2 : A_2 \vdash P :: y : C \otimes D$$

$$\emptyset \vdash Q :: y : A \multimap B$$

Q offers an input along y , no requirements from its environment

$$x : A \otimes B, y : C \otimes D \vdash R :: z : 1$$

R does not offer visible behavior z and yet depends on two different output behaviors from its environment

Propositions as Session Types: $A \otimes B$

We have some decisions to make:

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash ?? :: ?? : A \otimes B}$$

$$\frac{\Delta, y : A, x : B \vdash R :: z : C}{\Delta, ?? : A \otimes B \vdash ?? :: z : C}$$

Propositions as Session Types: $A \otimes B$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) (\bar{x} \langle y \rangle . (P \mid Q)) :: x : A \otimes B}$$

Propositions as Session Types: $A \otimes B$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) (\overline{x} \langle y \rangle . (P \mid Q)) :: x : A \otimes B}$$

Send y over x

Propositions as Session Types: $A \otimes B$

Execute P and Q in parallel

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) (\bar{x} \langle y \rangle . (P \mid Q)) :: x : A \otimes B}$$

Send y over x

Propositions as Session Types: $A \otimes B$

Execute P and Q in parallel

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) (\bar{x}\langle y \rangle . (P \mid Q)) :: x : A \otimes B}$$

Declare channel y as **local**, i.e., private

Send y over x

Propositions as Session Types: $A \otimes B$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) (\bar{x} \langle y \rangle . (P \mid Q)) :: x : A \otimes B}$$

$$\frac{\Delta, y : A, x : B \vdash R :: z : C}{\Delta, x : A \otimes B \vdash x(y).R :: z : C}$$

To **use** a ' $\otimes$ ', receive a name on x

Propositions as Session Types: $A \multimap B$

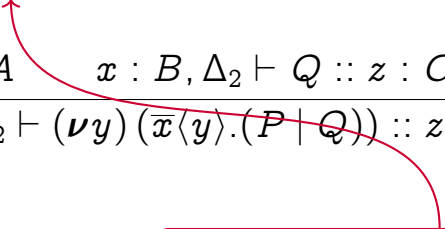
For $A \multimap B$, we have a symmetric situation:

$$\frac{\Delta, y : A \vdash P :: z : B}{\Delta \vdash z(y).P :: z : A \multimap B}$$

$$\frac{\Delta_1 \vdash P :: y : A \quad x : B, \Delta_2 \vdash Q :: z : C}{\Delta_1, x : A \multimap B, \Delta_2 \vdash (\nu y) (\bar{x}\langle y \rangle. (P \mid Q)) :: z : C}$$

Propositions as Session Types: $A \multimap B$

For $A \multimap B$, we have a symmetric situation:

$$\frac{\Delta, y : A \vdash P :: z : B}{\Delta \vdash z(y).P :: z : A \multimap B}$$
$$\frac{\Delta_1 \vdash P :: y : A \quad x : B, \Delta_2 \vdash Q :: z : C}{\Delta_1, x : A \multimap B, \Delta_2 \vdash (\nu y) (\bar{x}\langle y \rangle. (P \mid Q)) :: z : C}$$


To **offer** a ' $\multimap$ ', we implement a receive

Propositions as Session Types: $A \multimap B$

For $A \multimap B$, we have a symmetric situation:

$$\frac{\Delta, y : A \vdash P :: z : B}{\Delta \vdash z(y).P :: z : A \multimap B}$$
$$\frac{\Delta_1 \vdash P :: y : A \quad x : B, \Delta_2 \vdash Q :: z : C}{\Delta_1, x : A \multimap B, \Delta_2 \vdash (\nu y) (\bar{x}\langle y \rangle. (P \mid Q)) :: z : C}$$

To **use** a ' $\multimap$ ', we implement a send

To **offer** a ' $\multimap$ ', we implement a receive

Propositions as Session Types: The Unit 1 and Axiom

More decisions to make:

$$\emptyset \vdash ?? :: x : 1 \qquad \frac{\Delta \vdash Q :: z : C}{\Delta, ?? : 1 \vdash ?? :: z : C} \qquad x : A \vdash ?? :: y : A$$

Propositions as Session Types: The Unit 1 and Axiom

$$\frac{}{\emptyset \vdash \overline{x}\langle \rangle :: x : 1} \quad \frac{\Delta \vdash Q :: z : C}{\Delta, x : 1 \vdash x().Q :: z : C} \quad x : A \vdash [y \leftarrow x] :: y : A$$

Propositions as Session Types: The Unit 1 and Axiom

Close the channel x

$$\frac{}{\emptyset \vdash \overline{x} \langle \rangle :: x : 1} \quad \frac{\Delta \vdash Q :: z : C}{\Delta, x : 1 \vdash x().Q :: z : C} \quad x : A \vdash [y \leftarrow x] :: y : A$$

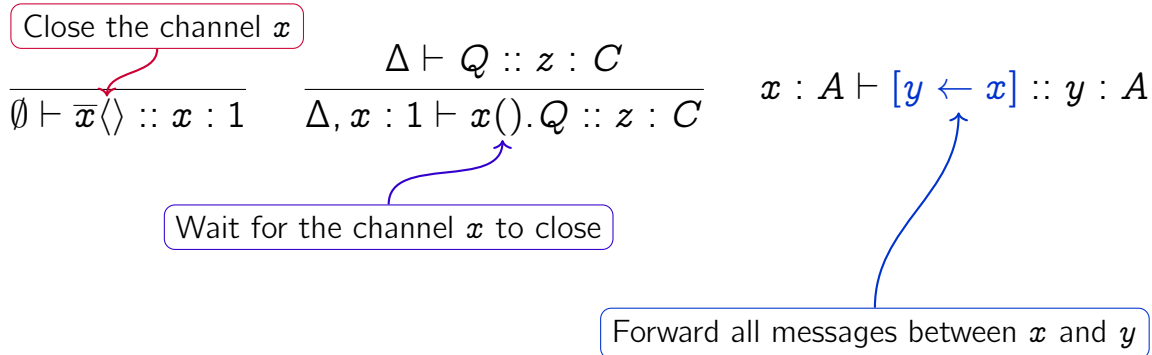
Propositions as Session Types: The Unit 1 and Axiom

Close the channel x

$$\frac{}{\emptyset \vdash \bar{x}\langle \rangle :: x : 1} \quad \frac{\Delta \vdash Q :: z : C}{\Delta, x : 1 \vdash x().Q :: z : C} \quad x : A \vdash [y \leftarrow x] :: y : A$$

Wait for the channel x to close

Propositions as Session Types: The Unit 1 and Axiom



Propositions as Session Types: An Alternative for Unit 1

We have just seen an explicit interpretation of 1. There is also the so-called **silent** interpretation:

0 is the process that does nothing

$$\frac{}{\emptyset \vdash 0 :: x : 1}$$


$$\frac{\Delta \vdash Q :: z : C}{\Delta, x : 1 \vdash Q :: z : C}$$


No explicit action

Interpreting LL Propositions as Session Types

end	terminate the session	1
$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$U \multimap S$

Interpreting LL Propositions as Session Types

end	terminate the session	1
$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$U \multimap S$
$S_1 \oplus S_2$	select one between S_1 (left) and S_2 (right)	idem
$S_1 \& S_2$	offer the alternatives S_1 (left) and S_2 (right)	idem

Interpreting LL Propositions as Session Types

end	terminate the session	1
$!U; S$	send value of type U , continue as S	$U \otimes S$
$?U; S$	receive value of type U , continue as S	$U \multimap S$
$S_1 \oplus S_2$	select one between S_1 (left) and S_2 (right)	idem
$S_1 \& S_2$	offer the alternatives S_1 (left) and S_2 (right)	idem
$\oplus\{l_1 : S_1, \dots, l_n : S_n\}$	select one between $S_1, \dots, S_n$	“idem”
$\&\{l_1 : S_1, \dots, l_n : S_n\}$	offer the alternatives $S_1, \dots, S_n$	“idem”

Propositions as Session Types: Additive Conjunction

Binary operators:

$$\frac{\Delta \vdash P :: x : A \quad \Delta \vdash Q :: x : B}{\Delta \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B}$$

$$\frac{\Delta, x : A \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inl}; Q :: z : C}$$

$$\frac{\Delta, x : B \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inr}; Q :: z : C}$$

Notice: ‘ $\triangleleft$ ’ means sending a label and ‘ $\triangleright$ ’ means receiving a label.

Propositions as Session Types: Additive Conjunction

The generalization to n -ary operators:

$$\frac{\Delta \vdash P_i :: x : A_i}{\Delta \vdash x \triangleright \{1_1 : P_1, \dots, 1_n : P_n\} :: x : \&\{1_i : A_i\}_{1 \leq i \leq n}}$$

$$\frac{\Delta, x : A_i \vdash Q :: z : C}{\Delta, x : \&\{1_i : A_i\} \vdash x \triangleleft 1_i; Q :: z : C}$$

Propositions as Session Types: Additive Conjunction

Let's examine first at the binary version:

Branch on x : proceed either as P or Q

$$\frac{\Delta \vdash P :: x : A \quad \Delta \vdash Q :: x : A}{\Delta \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B}$$


$$\frac{\Delta, x : A \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inl}; Q :: z : C}$$

$$\frac{\Delta, x : B \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inr}; Q :: z : C}$$

Propositions as Session Types: Additive Conjunction

Let's examine first at the binary version:

Branch on x : proceed either as P or Q

$$\frac{\Delta \vdash P :: x : A \quad \Delta \vdash Q :: x : A}{\Delta \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B}$$

$$\frac{\Delta, x : A \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inl}; Q :: z : C}$$

$$\frac{\Delta, x : B \vdash Q :: z : C}{\Delta, x : A \& B \vdash x \triangleleft \text{inr}; Q :: z : C}$$

Select either left or right session continuation

Propositions as Session Types: Additive Conjunction

Let's look now at the generalized version:

Branch on x : proceed as one of the P_i

$$\frac{\Delta \vdash P_i :: x : A_i}{\Delta \vdash x \triangleright \{l_1 : P_1, \dots, l_n : P_n\} :: x : \&\{l_i : A_i\}_{1 \leq i \leq n}}$$

$$\frac{\Delta, x : A_i \vdash Q :: z : C}{\Delta, x : \&\{l_i : A_i\} \vdash x \triangleleft l_i; Q :: z : C}$$

Propositions as Session Types: Additive Conjunction

Let's look now at the generalized version:

Branch on x : proceed as one of the P_i

$$\frac{\Delta \vdash P_i :: x : A_i}{\Delta \vdash x \triangleright \{l_1 : P_1, \dots, l_n : P_n\} :: x : \&\{l_i : A_i\}_{1 \leq i \leq n}}$$

$$\frac{\Delta, x : A_i \vdash Q :: z : C}{\Delta, x : \&\{l_i : A_i\} \vdash x \triangleleft l_i; Q :: z : C}$$

Select exactly one of the session continuations

Propositions as Session Types: Additive Disjunction

For $A \oplus B$, we have a symmetric situation:

$$\frac{\Delta \vdash P :: x : A}{\Delta \vdash x \triangleleft \text{inl}; P :: x : A \oplus B} \qquad \frac{\Delta \vdash Q :: x : B}{\Delta \vdash x \triangleleft \text{inr}; Q :: x : A \oplus B}$$

$$\frac{\Delta, x : A \vdash P :: z : C \quad \Delta, x : B \vdash Q :: z : C}{\Delta, x : A \oplus B \vdash x \triangleright \{\text{inl} : P; \text{inr} : Q\} :: z : C}$$

Propositions as Session Types: Additive Disjunction

For $A \oplus B$, we have a symmetric situation:

$$\frac{\Delta \vdash P :: x : A}{\Delta \vdash x \triangleleft \text{inl}; P :: x : A \oplus B} \qquad \frac{\Delta \vdash Q :: x : B}{\Delta \vdash x \triangleleft \text{inr}; Q :: x : A \oplus B}$$
$$\frac{\Delta, x : A \vdash P :: z : C \quad \Delta, x : B \vdash Q :: z : C}{\Delta, x : A \oplus B \vdash x \triangleright \{\text{inl} : P; \text{inr} : Q\} :: z : C}$$

To **offer** a ' $\oplus$ ', we implement a selection

Propositions as Session Types: Additive Disjunction

For $A \oplus B$, we have a symmetric situation:

$$\frac{\Delta \vdash P :: x : A}{\Delta \vdash x \triangleleft \text{inl}; P :: x : A \oplus B}$$

$$\frac{\Delta \vdash Q :: x : B}{\Delta \vdash x \triangleleft \text{inr}; Q :: x : A \oplus B}$$

$$\frac{\Delta, x : A \vdash P :: z : C \quad \Delta, x : B \vdash Q :: z : C}{\Delta, x : A \oplus B \vdash x \triangleright \{\text{inl} : P; \text{inr} : Q\} :: z : C}$$



To **use** a ' $\oplus$ ', we implement a branching

The Process Language, Up to Here

A variant of the **π -calculus** (Milner, Parrow & Walker, 1992):

$P, Q ::=$	$[y \leftarrow x]$	forwarder between sessions x and y
	$(\nu y) (\bar{x}\langle y \rangle. (P \mid Q))$	send y over x , then execute P and Q
	$x(y).P$	receive y over x , then execute P
	$\bar{x}\langle \rangle$	close session x
	$x().P$	wait-close session x , then execute P
	$x \triangleleft l_i; P$	select label l_i along x , then execute P
	$x \triangleright \{l_1 : P_1, \dots, l_n : P_n\}$	branch on x , offering labels $l_1, \dots, l_n$
	0	inaction (for the silent interpretation)

What are we missing?

Propositions as Session Types: Cut

$$\frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash ?? :: z : C}$$

Propositions as Session Types: Cut

P can provide A along x

Q relies on x along A to provide $z : C$

$$\frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash ?? :: z : C}$$

Propositions as Session Types: Cut

P can provide A along x

Q relies on x along A to provide $z : C$


$$\frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

Propositions as Session Types: Cut

P can provide A along x

Q relies on x along A to provide $z : C$

$$\frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

Execute P and Q in parallel

Propositions as Session Types: Cut

P can provide A along x

Q relies on x along A to provide $z : C$

$$\frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

Execute P and Q in parallel

Declare channel x **local** to P and Q

The Process Language, Now With Concurrency

$P, Q ::=$	$[y \leftarrow x]$	forwarder between sessions x and y
	$(\nu y)(\bar{x}\langle y \rangle.(P \mid Q))$	send y over x , then execute P and Q
	$x(y).P$	receive y over x , then execute P
	$\bar{x}\langle \rangle$	close session x
	$x().P$	wait-close session x , then execute P
	$x \triangleleft 1_i; P$	select label 1_i along x , then execute P
	$x \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}$	branch on x , offering labels $1_1, \dots, 1_n$
	0	inaction (silent interpretation)
	$(\nu x)(P \mid Q)$	parallel composition of P and Q on x

- In $(\nu y)P$ and $x(y).P$, name y is *bound* with scope P .
- We write $P\{y/z\}$ to denote the **substitution** of z for y in P .

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces”
 $x_1, \dots, x_n$.

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

$$x_1 : A_1, x_2 : A_2, \dots, x_n : A_n \vdash P :: y : A$$

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

$$x_2 : A_2, \dots, x_n : A_n \vdash (\nu x_1)(P \mid R_1) :: y : A$$

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

$$x_3 : A_3, \dots, x_n : A_n \vdash (\nu x_2)((\nu x_1)(P \mid R_1) \mid R_2) :: y : A$$

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

$$\emptyset \vdash (\nu x_n)(\dots(\nu x_2)((\nu x_1)(P \mid R_1) \mid R_2) \dots \mid R_n) :: y : A$$

where processes R_i offer A_i on x_i .

Open and Closed Systems

- Generally speaking, if we have a process P with judgment

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: y : A$$

P is an **open** system: it can be composed with other processes, via “interfaces” $x_1, \dots, x_n$.

- A process such as $\emptyset \vdash Q :: y : A$ is a **closed** system: its only “interface” is y .
- The interpretation of cut as typed composition gives an immediate recipe for closing systems. Example:

$$\emptyset \vdash (\nu x_n)(\dots(\nu x_2)((\nu x_1)(P \mid R_1) \mid R_2) \dots \mid R_n) :: y : A$$

where processes R_i offer A_i on x_i .

- The distinction between open and closed is key in the theory of processes in general, and in the meta-theory of the interpretation in particular.

Processes for The Two-Buyer Protocol

- Recall Bob's involvement in the two-buyer protocol:

$$?cost; \oplus \begin{cases} \text{share} : & !address; ?ok; \text{end} \\ \text{close} : & ?bye; \text{end} \end{cases}$$

- Here's a possible process implementation for Bob in our process language:

$$\text{Bob} = b(y).b \triangleleft \text{share}; (\nu a)\bar{b}\langle a \rangle.([a \leftarrow a'] \mid b(u).0)$$

(This is the silent interpretation!)

Processes for The Two-Buyer Protocol

- Recall Bob's involvement in the two-buyer protocol:

$$?cost; \oplus \begin{cases} \text{share} : !\text{address}; ?ok; \text{end} \\ \text{close} : ?\text{bye}; \text{end} \end{cases}$$

- Here's a possible process implementation for Bob in our process language:

$$\text{Bob} = b(y).b \triangleleft \text{share}; (\nu a)\bar{b}\langle a \rangle.([a \leftarrow a'] \mid b(u).0)$$

(This is the silent interpretation!)

- Process Bob is **well-typed**, as the following judgment is provable:

$$a' : A \vdash \text{Bob} :: b : C \multimap \underbrace{\oplus\{\text{share} : A \otimes (\text{OK} \multimap 1) ; \text{close} : 1\}}_{\text{bobProto}}$$

where, for simplicity, $C = A = \text{OK} = 1$.

Processes for The Two-Buyer Protocol

- Let us now consider an implementation for Alice. She is involved in two different sessions, on which she depends, so we expect a judgment:

$$b : \text{bobProto}, s : \text{sellerProto} \vdash \text{Alice} :: z : 1$$

- We can define the process Alice, which manages both sessions:

$$\bar{s}\langle \text{book} \rangle . s(q) . \bar{b}\langle q \rangle . b \triangleright \left\{ \begin{array}{l} \text{share} : b(addr) . s \triangleleft \text{buy} ; \bar{s}\langle p \rangle . \bar{s}\langle addr \rangle . \bar{b}\langle ok \rangle . 0 \\ \text{close} : s \triangleleft \text{cancel} ; \bar{b}\langle bye \rangle . \bar{s}\langle exit \rangle . 0 \end{array} \right\}$$

- This way, the complete (closed) system would look as follows:

$$(\nu b)((\nu s)(\text{Alice} \mid \text{Seller}) \mid \text{Bob})$$

Outline

Preliminaries

Computational Interpretation of LL: Statics

- Sequent Calculus

- Output and Input

- Unit and Axiom

- Additives

- Cut

- Processes

Dynamics

- Cut Reduction

- Process Reduction

- Properties

Proof Simplification and Process Semantics

- Up to here, we have seen how to interpret propositions as session types, and how to extract processes from proofs.
- This is only half of the expected correspondence: We still need to see how proof simplification corresponds to the semantics of processes.

Proof Simplification and Process Semantics

Proof transformations have different consequences on processes:

- Principal cut reductions induce **process reduction**, denoted $\longrightarrow$, a relation that defines the behavior of a process on its own (i.e. synchronizations)
- Some proof transformations correspond to **structural congruence**, denoted $\equiv$, a relation that describes syntactic rearrangements for processes
- Commuting conversions do not have precise correspondences, but can be explained via **behavioral equivalences**

Principal Cut Reductions: Synchronization via $\otimes$ (1/4)

$$\frac{\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) \bar{x} \langle y \rangle . (P \mid Q) :: x : A \otimes B} \quad \frac{\Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_3, x : A \otimes B \vdash x(y) . R :: z : C}}{\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) \left((\nu y) \bar{x} \langle y \rangle . (P \mid Q) \mid x(y) . R \right) :: z : C}$$

$\longrightarrow$

Principal Cut Reductions: Synchronization via $\otimes$ (2/4)

$$\frac{\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) \bar{x} \langle y \rangle . (P \mid Q) :: x : A \otimes B} \quad \frac{\Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_3, x : A \otimes B \vdash x(y) . R :: z : C}}{\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) \left((\nu y) \bar{x} \langle y \rangle . (P \mid Q) \mid x(y) . R \right) :: z : C}$$

$\longrightarrow$

Principal Cut Reductions: Synchronization via $\otimes$ (3/4)

$$\begin{array}{c}
 \frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) \bar{x} \langle y \rangle . (P \mid Q) :: x : A \otimes B} \quad \frac{\Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_3, x : A \otimes B \vdash x(y) . R :: z : C} \\
 \hline
 \Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) \left((\nu y) \bar{x} \langle y \rangle . (P \mid Q) \mid x(y) . R \right) :: z : C \\
 \\
 \longrightarrow \\
 \frac{\Delta_1 \vdash P :: y : A \quad \Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_1, x : B, \Delta_3 \vdash (\nu y) (P \mid R) :: z : C} \\
 \hline
 \end{array}$$

Principal Cut Reductions: Synchronization via $\otimes$ (4/4)

$$\begin{array}{c}
 \frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash (\nu y) \bar{x} \langle y \rangle . (P \mid Q) :: x : A \otimes B} \quad \frac{\Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_3, x : A \otimes B \vdash x(y).R :: z : C} \\
 \hline
 \Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) \left((\nu y) \bar{x} \langle y \rangle . (P \mid Q) \mid x(y).R \right) :: z : C \\
 \\
 \longrightarrow \\
 \frac{\Delta_2 \vdash Q :: x : B \quad \frac{\Delta_1 \vdash P :: y : A \quad \Delta_3, y : A, x : B \vdash R :: z : C}{\Delta_1, x : B, \Delta_3 \vdash (\nu y) (P \mid R) :: z : C}}{\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) \left(Q \mid (\nu y) (P \mid R) \right) :: z : C}
 \end{array}$$

This way, we have extracted the following reduction on processes:

$$(\nu x) \left((\nu y) \bar{x} \langle y \rangle . (P \mid Q) \mid x(y).R \right) \longrightarrow (\nu x) \left(Q \mid (\nu y) (P \mid R) \right)$$

Principal Cut Reductions: Synchronization via $\multimap$

The case of $\multimap$ is similar. We have the following:

$$\frac{\frac{\Delta_1, y : A \vdash R :: x : B}{\Delta_1 \vdash x(y).R :: x : A \multimap B} \quad \frac{\Delta_2 \vdash P :: y : A \quad \Delta_3, x : B \vdash Q :: z : C}{\Delta_2, \Delta_3, x : A \multimap B \vdash (\nu y)\bar{x}\langle y \rangle.(P \mid Q) :: z : C}}{\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)(x(y).R \mid (\nu y)\bar{x}\langle y \rangle.(P \mid Q)) :: z : C}$$

which, omitting large bits of the derivation, can be simplified into

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)((\nu y)(P \mid R) \mid Q) :: z : C$$

That is, we have the following reduction on processes:

$$(\nu x)(x(y).R \mid (\nu y)\bar{x}\langle y \rangle.(P \mid Q)) \longrightarrow (\nu x)((\nu y)(P \mid R) \mid Q)$$

Where is My Communication?

In our rules, the name (on x) and the input parameter are the same (i.e. y).

In general, these names need not match and reduction involves a name substitution and scope extrusion. In the case of $\multimap$:

$$\begin{array}{c} \Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) (x(y).R \mid (\nu w) \bar{x} \langle w \rangle . (P \mid Q)) :: z : C \\ \longrightarrow \\ \Delta_1, \Delta_2, \Delta_3 \vdash (\nu x) ((\nu w) (P \mid R\{w/y\}) \mid Q) :: z : C \end{array}$$

Where is My Communication?

In our rules, the name (on x) and the input parameter are the same (i.e. y).

In general, these names need not match and reduction involves a name substitution and scope extrusion. In the case of $\multimap$:

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)(x(y).R \mid (\nu w)\bar{x}\langle w\rangle.(P \mid Q)) :: z : C$$

$\longrightarrow$

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)((\nu w)(P \mid R\{w/y\}) \mid Q) :: z : C$$

R contains occurrences of y

Where is My Communication?

In our rules, the name (on x) and the input parameter are the same (i.e. y).

In general, these names need not match and reduction involves a name substitution and scope extrusion. In the case of $\multimap$:

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)(x(y).R \mid (\nu w)\bar{x}\langle w\rangle.(P \mid Q)) :: z : C$$

$\longrightarrow$

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)((\nu w)(P \mid R\{w/y\}) \mid Q) :: z : C$$

R contains occurrences of y

Name w has been received by R

Where is My Communication?

In our rules, the name (on x) and the input parameter are the same (i.e. y).

In general, these names need not match and reduction involves a name substitution and scope extrusion. In the case of $\multimap$:

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)(x(y).R \mid (\nu w)\bar{x}\langle w\rangle.(P \mid Q)) :: z : C$$

$\longrightarrow$

$$\Delta_1, \Delta_2, \Delta_3 \vdash (\nu x)((\nu w)(P \mid R\{w/y\}) \mid Q) :: z : C$$

The scope of w has expanded!

Name w has been received by R

Process Reductions from Cut Reductions: Choice (1/3)

$$\frac{\frac{\Delta_1 \vdash P :: x : A \quad \Delta_1 \vdash Q :: x : A}{\Delta_1 \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B} \quad \frac{\Delta_2, x : A \vdash R :: z : C}{\Delta_2, x : A \& B \vdash x \triangleleft \text{inl}; R :: z : C}}{\Delta_1, \Delta_2 \vdash (\nu x) (x \triangleright \{\text{inl} : P, \text{inr} : Q\} \mid x \triangleleft \text{inl}; R) :: z : C} \longrightarrow$$

Process Reductions from Cut Reductions: Choice (2/3)

$$\frac{\frac{\Delta_1 \vdash P :: x : A \quad \Delta_1 \vdash Q :: x : A}{\Delta_1 \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B} \quad \frac{\Delta_2, x : A \vdash R :: z : C}{\Delta_2, x : A \& B \vdash x \triangleleft \text{inl}; R :: z : C}}{\Delta_1, \Delta_2 \vdash (\nu x) (x \triangleright \{\text{inl} : P, \text{inr} : Q\} \mid x \triangleleft \text{inl}; R) :: z : C} \longrightarrow \frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash R :: z : C}{}$$

Process Reductions from Cut Reductions: Choice (3/3)

$$\begin{array}{c}
 \frac{\Delta_1 \vdash P :: x : A \quad \Delta_1 \vdash Q :: x : A}{\Delta_1 \vdash x \triangleright \{\text{inl} : P, \text{inr} : Q\} :: x : A \& B} \quad \frac{\Delta_2, x : A \vdash R :: z : C}{\Delta_2, x : A \& B \vdash x \triangleleft \text{inl}; R :: z : C} \\
 \hline
 \Delta_1, \Delta_2 \vdash (\nu x) (x \triangleright \{\text{inl} : P, \text{inr} : Q\} \mid x \triangleleft \text{inl}; R) :: z : C \\
 \\
 \longrightarrow \\
 \\
 \frac{\Delta_1 \vdash P :: x : A \quad \Delta_2, x : A \vdash R :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x) (P \mid R)}
 \end{array}$$

This way, we have the following reduction on processes:

$$(\nu x) (x \triangleright \{\text{inl} : P, \text{inr} : Q\} \mid x \triangleleft \text{inl}; R) \longrightarrow (\nu x) (P \mid R)$$

Semantics of Session-typed Processes: Summary

The relation $\longrightarrow$ on processes is defined via principal cut reductions, as follows:

$$\begin{aligned}(\nu x)\big((\nu y)\bar{x}\langle y\rangle.(P_1 \mid P_2) \mid x(z).Q\big) &\longrightarrow (\nu x)\big(P_2 \mid (\nu y)(P_1 \mid Q\{y/z\})\big) \\(\nu x)\big(x \triangleright \{1_i : P_i\}_{i \in I} \mid x \triangleleft 1_i; R\big) &\longrightarrow (\nu x)\big(P_i \mid R\big) \\(\nu x)\big(x \triangleleft 1_i; R \mid x \triangleright \{1_i : P_i\}_{i \in I}\big) &\longrightarrow (\nu x)\big(R \mid P_i\big) \\(\nu x)([x \leftarrow y] \mid P) &\longrightarrow P\{y/x\} \quad (y \text{ is not free in } P) \\P \longrightarrow P' &\implies (\nu x)(P \mid Q) \longrightarrow (\nu x)(P' \mid Q)\end{aligned}$$

All the reductions remove a cut, possibly introducing new cuts in the process.

Other Proof Transformations

- **Structural congruence**, noted $\equiv$, concerns “expected” properties of processes.
Examples (omitting side conditions):

$$\begin{aligned}(\nu x)((\nu y)(P \mid Q) \mid R) &\equiv (\nu y)(P \mid (\nu x)(Q \mid R)) \\(\nu x)(P \mid (\nu y)(Q \mid R)) &\equiv (\nu y)(Q \mid (\nu x)(P \mid R))\end{aligned}$$

Other Proof Transformations

- **Structural congruence**, noted $\equiv$, concerns “expected” properties of processes.
Examples (omitting side conditions):

$$\begin{aligned}(\nu x)((\nu y)(P \mid Q) \mid R) &\equiv (\nu y)(P \mid (\nu x)(Q \mid R)) \\(\nu x)(P \mid (\nu y)(Q \mid R)) &\equiv (\nu y)(Q \mid (\nu x)(P \mid R))\end{aligned}$$

- **Commuting conversions** induce “unusual” process equalities, denoted $\sim_{cc}$.
Examples (omitting side conditions):

$$\begin{aligned}x(u).y(w).P &\sim_{cc} y(w).x(u).P \\x(u).(\nu w)\bar{y}\langle w \rangle.(P_1 \mid P_2) &\sim_{cc} (\nu w)\bar{y}\langle w \rangle.(x(u).P_1 \mid P_2) \\x(u).(\nu y)(P_1 \mid P_2) &\sim_{cc} (\nu y)(x(u).P_1 \mid P_2)\end{aligned}$$

$\sim_{cc}$ can be justified via a (typed) bisimilarity on processes.

Properties of π DILL

Theorem (Subject Reduction)

If $\Delta \vdash P :: z : C$ and $P \longrightarrow Q$ then $\Delta \vdash Q :: z : C$

SR ensures our expectations about session fidelity and communication safety.

Properties of π DILL

Theorem (Subject Reduction)

If $\Delta \vdash P :: z : C$ and $P \longrightarrow Q$ then $\Delta \vdash Q :: z : C$

SR ensures our expectations about session fidelity and communication safety.

Theorem (Deadlock-freedom)

If $\Delta \vdash P :: z : C$ and $P \not\longrightarrow$ then P is blocked on either z or a channel from Δ

Corollary

If $\emptyset \vdash P :: z : 1$ and $P \not\longrightarrow$ then $P = \bar{z}\langle \rangle$.

More on Deadlock-Freedom / Progress

- Remarkably, the concurrent interpretation of LL leads to a type system that avoids stuck processes.

More on Deadlock-Freedom / Progress

- Remarkably, the concurrent interpretation of LL leads to a type system that avoids stuck processes.
- A paradigmatic example of a stuck process:

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\textcolor{blue}{z}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)z(\textcolor{blue}{u}).(\textcolor{red}{x}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

Note the circular dependency between the two processes in parallel.

More on Deadlock-Freedom / Progress

- Remarkably, the concurrent interpretation of LL leads to a type system that avoids stuck processes.
- A paradigmatic example of a stuck process:

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\textcolor{blue}{z}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)\textcolor{blue}{z}(\textcolor{blue}{u}).(\textcolor{red}{x}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

Note the circular dependency between the two processes in parallel.

- The following process does not have this dependency:

$$Q = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\textcolor{blue}{z}\langle \textcolor{blue}{w} \rangle.(Q_1 \mid Q_2) \\ \mid (\nu y)\textcolor{red}{x}\langle \textcolor{red}{y} \rangle.(\textcolor{blue}{z}(\textcolor{blue}{u}).Q_3 \mid Q_4))$$

Still, Q cannot be typed in the interpretation - can you see why?

Taking Stock

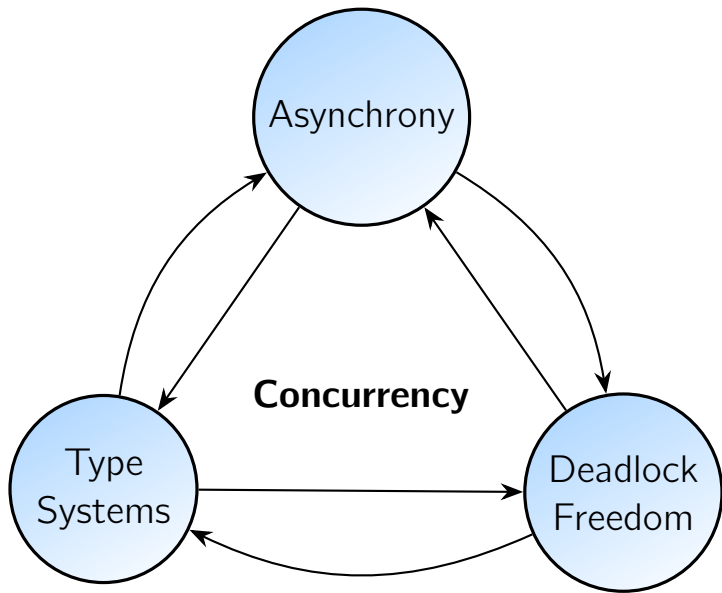
Today:

- Concurrent interpretation of LL: statics and dynamics
- Left and right rules per connective - rely and guarantee interactive behaviors
- Cut reductions and process synchronizations
- A look at the correctness properties ensured by the logic-based type system
- The computational interpretation of proof transformations
- Deadlock freedom (DF) and progress

Tomorrow:

- Asynchronous processes and DF

Asynchronous Communication



The Deadlock Freedom Property (DF)

- ▶ Informally, the guarantee that processes “never get stuck”.
- ▶ Even a single component that becomes permanently blocked awaiting a message can be problematic.
- ▶ Difficulty: Non-local component analysis.



Example

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

Example

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

This program follows its protocol ✓

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

Example

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

This program follows its protocol ✓

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

This program follows its protocol ✓

Example, with Concurrency

```
peter c1 c2 =  
  let (x, c1) = receive c1 in  
  let c2 = send (41) c2 in  
  wait c1;  
  close c2;  
  ()
```

```
miles c1 c2 =  
  let (x, c2) = receive c2 in  
  let c1 = send (42) c1 in  
  wait c2;  
  close c1;  
  ()
```

```
main =  
  let (c1, c1dual) = new @T () in  
  let (c2, c2dual) = new @T () in  
  fork (miles c1dual c2);  
  peter c1 c2dual
```

Is 'main' deadlock-free?



Using Processes to Enforce DF

Program

Processes (π -calculus)

Using Processes to Enforce DF

Program

Processes (π -calculus)

Type System (for DF)



Using Processes to Enforce DF

Program

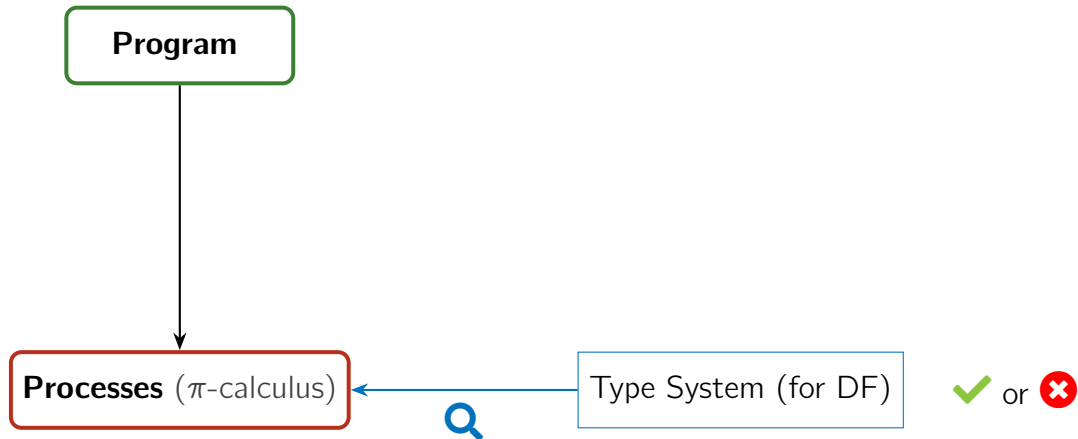
Processes (π -calculus)

Type System (for DF)

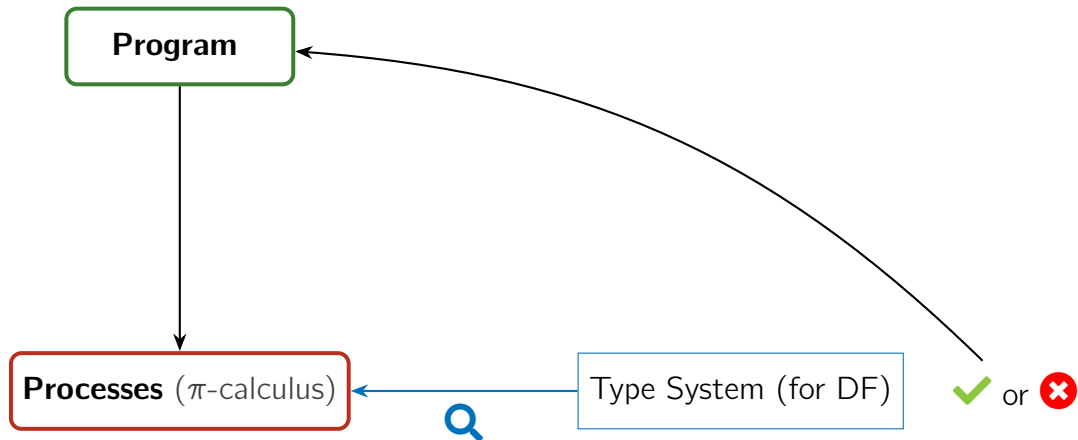


✓ or ✗

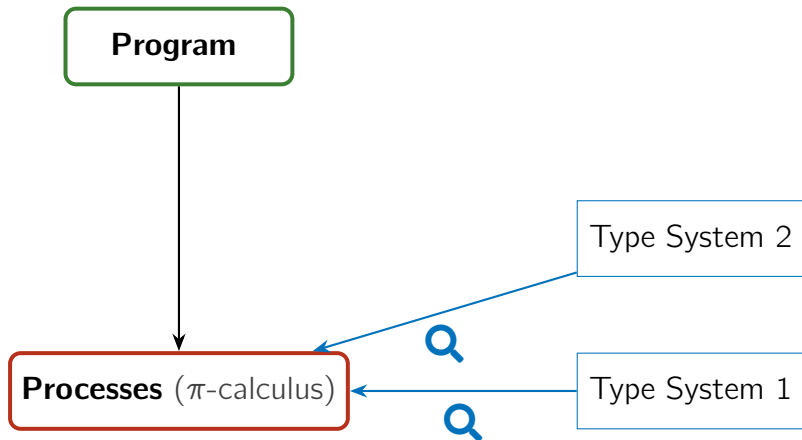
Using Processes to Enforce DF



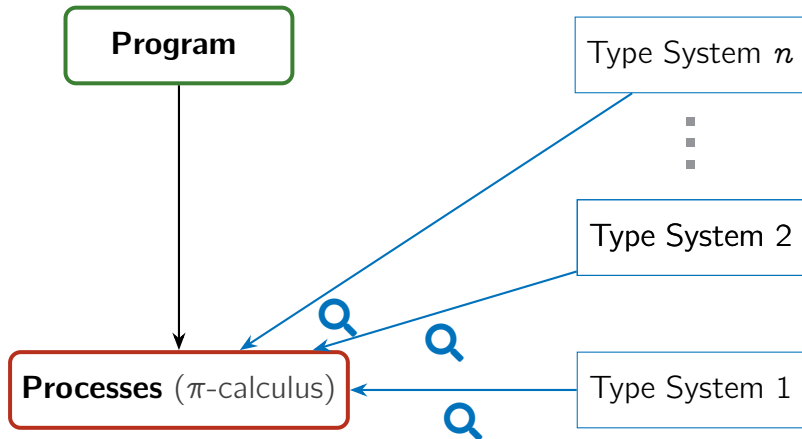
Using Processes to Enforce DF



Using Processes to Enforce DF: Many Different Type Systems!



Using Processes to Enforce DF: Many Different Type Systems!



Challenges

Design more advanced
type systems for DF

Contrast type systems
that enforce DF

Challenges

Design more advanced
type systems for DF



Contrast type systems
that enforce DF

Challenges

Design more advanced
type systems for DF



Contrast type systems
that enforce DF

Our Insight: Session type systems derived from Linear Logic ('Propositions as Sessions') offer a guiding reference for both!

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Today An overview of DF enforcement in three typed asynchronous π -calculi:

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Today An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no **DF enforcement**)

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Today An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no **DF enforcement**)
2. ACP: DF by typing for tree-like process topologies (**basic enforcement**)

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Today An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no DF enforcement)
2. ACP: DF by typing for tree-like process topologies (basic enforcement)
3. APCP: DF also for circular process topologies (advanced enforcement)

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Today An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no DF enforcement)
2. ACP: DF by typing for tree-like process topologies (basic enforcement)
3. APCP: DF also for circular process topologies (advanced enforcement)

Full details: Our LMCS and ICE'24 papers (with Bas van den Heuvel).

Outline

Asynchronous Processes

Asynchronous CP

Priorities for AP

Process Syntax

$P, Q ::= x[a, b]$	send
$x(y, z); P$	receive
$P \mid Q$	parallel
$(\nu xy)P$	restriction
0	inaction
$[x \leftrightarrow y]$	forwarder
$\dots$	

Process Syntax

$P, Q ::= x[a, b]$	send
$x(y, z); P$	receive
$P \mid Q$	parallel
$(\nu xy)P$	restriction
0	inaction
$[x \leftrightarrow y]$	forwarder
$\dots$	

Conventions:

- ▶ $a, b, c, \dots, x, y, z, \dots$ denote **names** (or **endpoints**)
- ▶ Early letters of the alphabet denote **objects** of output-like constructs.
- ▶ $\tilde{x}, \tilde{y}, \tilde{z}, \dots$ denote finite sequences of names.

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftrightarrow z] \mid P) \longrightarrow P\{z/y\}}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftrightarrow z] \mid P) \longrightarrow P\{z/y\}}$$

[red-sc]

$$\frac{P \equiv P' \quad P' \longrightarrow P' \quad Q' \equiv Q}{P \longrightarrow Q}$$

[red-res]

$$\frac{P \longrightarrow Q}{(\nu xy)P \longrightarrow (\nu xy)Q}$$

[red-par]

$$\frac{P \longrightarrow Q}{P \mid R \longrightarrow Q \mid R}$$

Derived Constructs / Syntactic Sugar (TBR)

$$\overline{x}[y] \cdot P \triangleq (\nu ya)(\nu zb)(x[a, b] \mid P\{z/x\})$$

$$x(y); P \triangleq x(y, z); P\{z/x\}$$

$$\overline{x} \triangleleft \ell \cdot P \triangleq (\nu zb)(x[b] \triangleleft \ell \mid P\{z/x\})$$

$$x \triangleright \{i : P_i\}_{i \in I} \triangleq x(z) \triangleright \{i : P_i\{z/x\}\}_{i \in I}$$

Note: We use ‘ $\cdot$ ’ to signify that output-like operations are non blocking.

Continuations Induce Session Protocols

$$\begin{aligned} P \triangleq (\nu zu) & \Big((\nu xy) \Big((\nu ax') (x[v_1, a] \mid x'[v_2, b]) \\ & \mid (\nu cz') (z[v_3, c] \mid y(w_1, y'); y'(w_2, y''); Q) \Big) \\ & \mid u(w_3, u'); R \Big) \end{aligned}$$

Continuations Induce Session Protocols

$$P \triangleq (\nu zu) \Big((\nu xy) \Big((\nu \textcolor{brown}{a} \textcolor{brown}{x}') (x[v_1, \textcolor{brown}{a}] \mid \textcolor{brown}{x}'[v_2, b]) \\ \mid (\nu cz') (z[v_3, c] \mid y(w_1, \textcolor{brown}{y}'); \textcolor{brown}{y}'(w_2, y''); Q) \Big) \\ \mid u(w_3, u'); R \Big)$$

Using a **sugared syntax**:

$$P = (\nu zu) ((\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_1); y(w_2); Q') \mid u(w_3); R')$$

Continuations Induce Session Protocols

$$P \triangleq (\nu zu) \Big((\nu xy) \Big((\nu \textcolor{red}{ax}') (x[v_1, \textcolor{red}{a}] \mid \textcolor{red}{x}'[v_2, b]) \\ \mid (\nu cz') (z[v_3, c] \mid y(w_1, \textcolor{red}{y}'); \textcolor{red}{y}'(w_2, y''); Q) \Big) \\ \mid u(w_3, u'); R \Big)$$

Using a **sugared syntax**:

$$P = (\nu zu) ((\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_1); y(w_2); Q') \mid u(w_3); R')$$

We have independent reductions:

$$P \longrightarrow (\nu zu) ((\nu xy) (\bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_2); Q'\{v_1/w_1\}) \mid u(w_3); R')$$

$$P \longrightarrow (\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid y(w_1); y(w_2); Q') \mid R'\{v_3/w_3\}$$

Note: There is no reduction involving the send on x' : because x' is connected to the continuation of the send on x , it is thus not (yet) paired with a dual receive.

A Deadlocked Process

The “hello world” of deadlocked message-passing processes:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Process is stuck:

- ▶ The input on x is waiting for the right output on y ...
- ▶ ...the output on y is blocked by a receive (on w) waiting for the left output on u , which is blocked by the input on x .

A Deadlocked Process

The “hello world” of deadlocked message-passing processes:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Process is stuck:

- ▶ The input on x is waiting for the right output on y ...
- ▶ ...the output on y is blocked by a receive (on w) waiting for the left output on u , which is blocked by the input on x .

There is a **cyclic dependency** between the two threads.

Session Types

Types specify protocols. They (mostly) correspond to Linear Logic propositions:

$$\begin{array}{lcl} A, B ::= A \otimes B & & \text{send} \\ | A \wp B & & \text{receive} \\ | \bullet & & \text{closed protocol} \\ | \dots & & \end{array}$$

Session Types

Types specify protocols. They (mostly) correspond to Linear Logic propositions:

$$\begin{array}{ll} A, B ::= A \otimes B & \text{send} \\ | A \wp B & \text{receive} \\ | \bullet & \text{closed protocol} \\ | \dots & \end{array}$$

The **dual** of session type A , denoted $\overline{A}$:

$$\overline{A \otimes B} \triangleq \overline{A} \wp \overline{B}$$

$$\overline{A \wp B} \triangleq \overline{A} \otimes \overline{B}$$

$$\overline{\bullet} \triangleq \bullet$$

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

The context Γ disallows

- ▶ *weakening* (all assignments must be used, except names typed with $\bullet$)
- ▶ *contraction* (assignments may not be duplicated).

but obeys *exchange* (assignments may be silently reordered).

The empty context is written $\emptyset$.

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

[typ-par]

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta}$$

[typ-res]

$$\frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma}$$

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

[typ-par]

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta}$$

[typ-res]

$$\frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma}$$

[typ-fwd]

$$\frac{}{[x \leftrightarrow y] \vdash x : \overline{A}, y : A}$$

[typ-end]

$$\frac{P \vdash \Gamma}{P \vdash \Gamma, x : \bullet}$$

[typ-inact]

$$\frac{}{0 \vdash \emptyset}$$

Properties of AP

Well-typed processes respect their session protocols under reduction:

Theorem (Type Preservation for AP)

Given $P \vdash \Gamma$ and Q such that $P \equiv Q$ or $P \longrightarrow Q$, we have $Q \vdash \Gamma$.

Deadlocks are Typable in AP

Recall the process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

It can be typed as follows:

$$\begin{array}{c}
 \frac{}{u[a, b] \vdash u : \bullet \otimes \bullet, a : \bullet, b : \bullet} \\
 \hline
 u[a, b] \vdash u : \bullet \otimes \bullet, \\
 \quad v : \bullet, x' : \bullet, a : \bullet, b : \bullet \\
 \hline
 x(v, x'); u[a, b] \vdash x : \bullet \wp \bullet, \\
 \quad u : \bullet \otimes \bullet, \\
 \quad a : \bullet, b : \bullet \\
 \hline
 \vdots \\
 w(z, w'); y[c, d] \vdash w : \bullet \wp \bullet, \\
 \quad y : \bullet \otimes \bullet, \\
 \quad c : \bullet, d : \bullet \\
 \hline
 x(v, x'); u[a, b] \mid w(z, w'); y[c, d] \vdash x : \bullet \wp \bullet, u : \bullet \otimes \bullet, a : \bullet, b : \bullet, \\
 \quad w : \bullet \wp \bullet, y : \bullet \otimes \bullet, c : \bullet, d : \bullet \\
 \hline
 (\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d]) \vdash a : \bullet, b : \bullet, c : \bullet, d : \bullet
 \end{array}$$

[typ-end]²

[typ-recv]

[typ-par]

[typ-res]²

Outline

Asynchronous Processes

Asynchronous CP

Priorities for AP

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]$$

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\boxed{\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]}$$

- ▶ ACP guarantees DF by ruling out all possible cyclic dependencies: sub-processes can connect along **exactly one pair of names**.

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\boxed{\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]}$$

- ▶ ACP guarantees DF by ruling out all possible cyclic dependencies: sub-processes can connect along **exactly one pair of names**.
- ▶ Note: We write $\textcolor{teal}{c}\vdash$ instead of $\vdash$ to distinguish the two type systems.

The Deadlocked Process, Revisited

- The deadlocked process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

is not typable under $\mathcal{C}\vdash$:

its parallel sub-processes are connected on two pairs of names.

The Deadlocked Process, Revisited

- The deadlocked process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

is not typable under $\textcolor{blue}{c}\vdash$:

its parallel sub-processes are connected on two pairs of names.

- A well-typed variant, obtained by ‘parallelizing’ the right sub-process:

$$(\nu xy)\big((\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); 0) \mid y[c, d]\big) \textcolor{blue}{c}\vdash \begin{array}{l} a : \bullet, b : \bullet, \\ c : \bullet, d : \bullet \end{array}$$

Properties of ACP

As in AP, well-typed processes respect their protocols:

Theorem (Type Preservation for ACP)

Given $P \text{ }^c\vdash \Gamma$ and Q such that $P \text{ }^c\equiv Q$ or $P \text{ }^c\longrightarrow Q$, we have $Q \text{ }^c\vdash \Gamma$.

Thanks to the cut rule, we now also have:

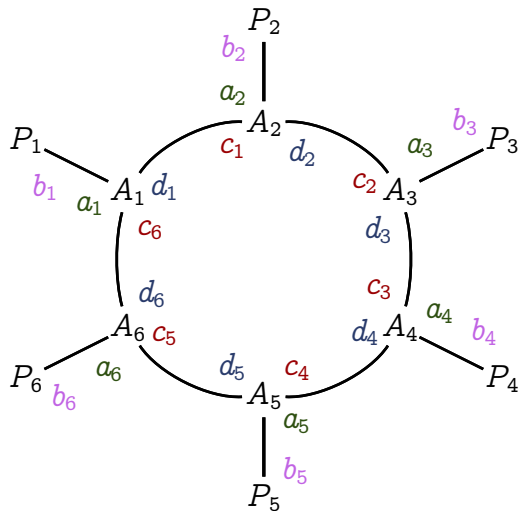
Theorem (Deadlock Freedom for ACP)

Given $P \text{ }^c\vdash \emptyset$, if $P \text{ }^c\nrightarrow$, then $P \text{ }^c\equiv 0$.

Not Typable in ACP: Milner's Cyclic Scheduler

Example of size 6:

$$\begin{aligned}
 &(\nu c_1 d_1) \cdots (\nu c_6 d_6) \Big((\nu a_1 b_1)(A_1 \mid P_1) \mid \\
 &\quad (\nu a_2 b_2)(A_2 \mid P_2) \mid \\
 &\quad \vdots \\
 &\quad (\nu a_6 b_6)(A_6 \mid P_6) \Big)
 \end{aligned}$$



Outline

Asynchronous Processes

Asynchronous CP

Priorities for AP

APCP: Priority-Based DF Enforcement

Asynchronous **Priority-Based** Classical Processes:

- ▶ Back to **separate rules** for parallel composition and restriction (as in AP)
- ▶ Excluding cyclic dependencies using **priorities**

The Laws of Priorities

Key Idea

Typing ensures that each action has a corresponding priority (its 'urgency').
actions with **lower priority** must not be blocked by actions with **higher priority**.

The Laws of Priorities

Key Idea

Typing ensures that each action has a corresponding priority (its 'urgency').
actions with **lower priority** must not be blocked by actions with **higher priority**.

Write $\pi, \rho, \dots$ to denote priorities (integers). Laws enforced by typing:

1. Outputs with priority π send messages with **larger priority** than π ;
2. A action with priority π are prefixed by inputs with **smaller priority** than π ;
3. Dual actions must have **equal priorities**.

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.
- For session type A , $pr(A)$ denotes its *priority*:

$$\begin{aligned} pr(A \otimes^{\pi} B) &\triangleq pr(A \wp^{\pi} B) \triangleq \pi \\ pr(\bullet) &\triangleq \omega \end{aligned}$$

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.
- For session type A , $pr(A)$ denotes its *priority*:

$$\begin{aligned} pr(A \otimes^{\pi} B) &\triangleq pr(A \wp^{\pi} B) \triangleq \pi \\ pr(\bullet) &\triangleq \omega \end{aligned}$$

- Duality: $A = \overline{B}$ iff (i) actions in A and B are complementary (as before) and (ii) their priorities coincide.

Typing rules of APCP: AP with Priorities

[typ-send]

$$\frac{\pi < pr(A), pr(B)}{x[y, z] \text{ }^P\vdash x : A \otimes^\pi B, y : \overline{A}, z : \overline{B}}$$

[typ-recv]

$$\frac{P \text{ }^P\vdash \Gamma, y : A, z : B \quad \pi < pr(\Gamma)}{x(y, z); P \text{ }^P\vdash \Gamma, x : A \wp^\pi B}$$

(Other rules similar or unchanged, with the extended duality)

Properties of APCP

Theorem (Type Preservation for APCP)

Given $P \vdash^P \Gamma$ and Q such that $P \equiv Q$ or $P \longrightarrow Q$, we have $Q \vdash^P \Gamma$.

Theorem (Deadlock Freedom for APCP)

Given $P \vdash^P \emptyset$, if $P \not\rightarrow$, then $P \equiv 0$.

Examples

- Recall the deadlocked process; it is rejected under $\textcolor{blue}{P} \vdash$:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Suppose that actions on x and y have priority $\textcolor{brown}{\pi}$, and that actions on u and w have priority $\textcolor{brown}{\rho}$. Using Rule [typ-recv] we require $\textcolor{brown}{\pi} < \textcolor{brown}{\rho}$ and $\textcolor{brown}{\rho} < \textcolor{brown}{\pi}$.

Examples

- Recall the deadlocked process; it is rejected under $\textcolor{blue}{P}\vdash$:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Suppose that actions on x and y have priority π , and that actions on u and w have priority ρ . Using Rule [typ-recv] we require $\pi < \rho$ and $\rho < \pi$.

- Recall the non-deadlocked variant:

$$(\nu xy)\big((\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); 0) \mid y[c, d]\big)$$

In this case, $\textcolor{blue}{P}\vdash$ only requires $\pi < \rho$ but not $\rho < \pi$, so no cyclic dependency is detected—the process is considered well typed.

Example

We construct a ring of three partial schedulers A_i , each of which simultaneously invokes a worker B_i and waits for the others to finish. Each scheduler A_i ($1 \leq i \leq 3$) communicates with its worker on name w_i (connected to the worker's name z_i), with its left neighbor on name x_{i-1} ($x_0 \triangleq x_3$), and with its right neighbor on name y_i . The schedulers and workers are defined as follows, writing ‘_’ for unused names:

$$A_1 \triangleq (\nu a_1 y'_1)(\nu b_1 w'_1)(\nu c_1 _) \\ (y_1[a_1] \triangleleft start \mid w_1[b_1] \triangleleft start \mid w'_1(_) \triangleright \{done : y'_1[c_1] \triangleleft done \\ \mid x_3(x'_3) \triangleright \{start : x'_3(_) \triangleright \{done : 0\}\}\})$$

$$A_i \triangleq (\nu a_i y'_i)(\nu b_i w'_i)(\nu c_i _) \\ (x_{i-1}(x'_{i-1}) \triangleright \{start : w_i[b_i] \triangleleft start \mid y_i[a_i] \triangleleft start \mid w'_i(_) \triangleright \{done : x'_{i-1}(_) \triangleright \{done : 0\}\}\}) \quad 2 \leq i < 3$$

$$B_i \triangleq (\nu d_i _)(z_i(z'_i) \triangleright \{start : doSomething(); z'_i[d_i] \triangleleft done\}) \quad 1 \leq i \leq 3$$

Example

Thus, A_1 starts the routine by signaling its right neighbor and worker. It then waits for its worker to finish, and signals its right neighbor it is done. Only in the end does it wait for a start signal from its left neighbor, and then wait for it to finish. The other two schedulers are defined identically. They wait for a start signal from their left neighbor, and then start their worker. After its worker is done, it signals to its right neighbor, and waits for its left neighbor to finish. Workers wait for a start signal, do some task (*doSomething()* stands for some arbitrary computation), and signal that they are finished afterwards.

A complete, cyclic scheduler $Sched$ is then formed by connecting the partial schedulers on channels between x_i and y_i for $1 \leq i \leq 3$.

$$Sched \triangleq (\nu x_1 y_1)(\nu x_2 y_2)(\nu x_3 y_3) \Big((\nu w_1 z_1)(A_1 \mid B_1) \mid (\nu w_2 z_2)(A_2 \mid B_2) \mid (\nu w_3 z_3)(A_3 \mid B_3) \Big)$$

Example

To type *Sched*, we assign the following types to names for $1 \leq i \leq 3$:

- ▶ $w_i : \oplus^{\pi_i} \{ \text{start} : \&^{\pi'_i} \{ \text{done} : \bullet \} \},$
- ▶ $x_i : \&^{\rho_i} \{ \text{start} : \&^{\rho'_i} \{ \text{done} : \bullet \} \},$
- ▶ $y_i : \oplus^{\sigma_i} \{ \text{start} : \oplus^{\sigma'_i} \{ \text{done} : \bullet \} \}$
- ▶ $z_i : \&^{\phi_i} \{ \text{start} : \oplus^{\phi'_i} \{ \text{done} : \bullet \} \}.$

Applications of Rule [typ-res] require that $\pi_i = \phi_i$, $\pi'_i = \phi'_i$, $\rho_i = \sigma_i$, and $\rho'_i = \sigma'_i$, for $1 \leq i \leq 3$. Applications of Rules [typ-sel] and [typ-bra] then require:

- ▶ $\rho_1 < \rho'_1$, and $\pi_1 < \pi'_1 < \rho'_1, \rho_3$;
- ▶ $\rho_1 < \pi_2, \rho_2, \pi'_2, \rho'_2$, $\pi_2 < \pi'_2$, $\rho_2 < \rho'_2$, and $\pi'_2 < \rho'_1, \rho'_2$;
- ▶ $\rho_2 < \pi_3, \rho_3, \pi'_3, \rho'_3$, $\pi_3 < \pi'_3$, $\rho_3 < \rho'_3$, and $\pi'_3 < \rho'_2, \rho'_3$.

We verify that these requirements are consistent, so $\text{Sched} \vdash^P \emptyset$. Hence, $\text{Sched} \in \vdash^P \setminus \vdash^C$, yet the process is deadlock free.

Application: DF in Concurrent Functional Sessions

LASTⁿ:
Concurrent functions
(can deadlock)

Application: DF in Concurrent Functional Sessions

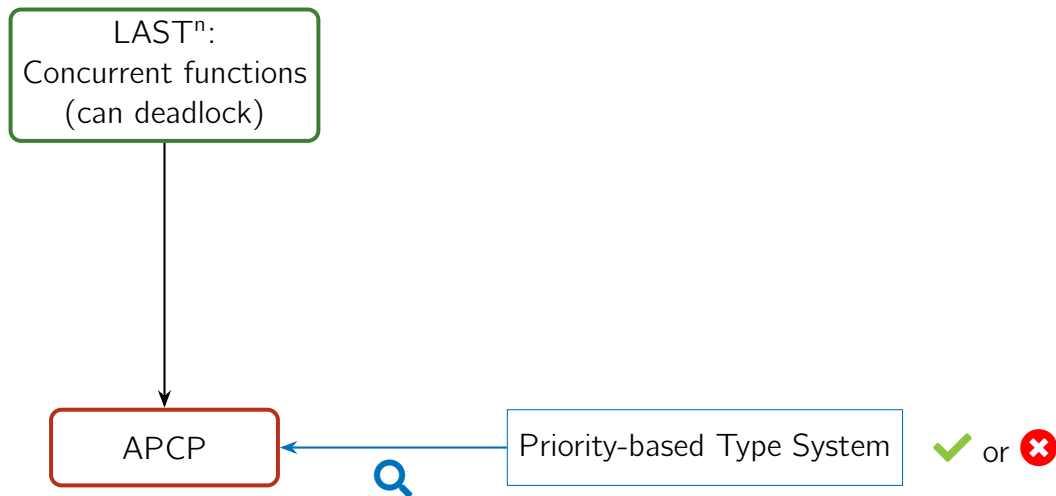
LASTⁿ:
Concurrent functions
(can deadlock)

APCP

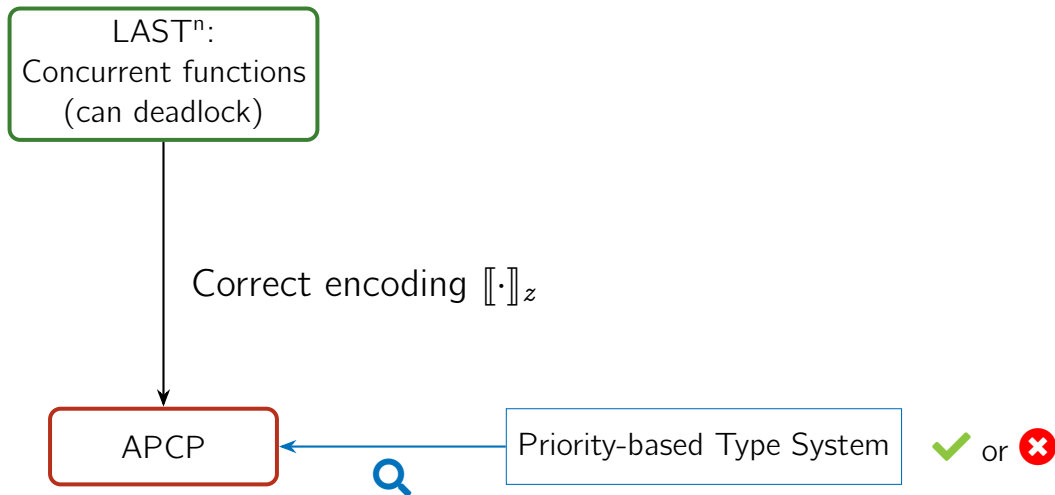
Priority-based Type System



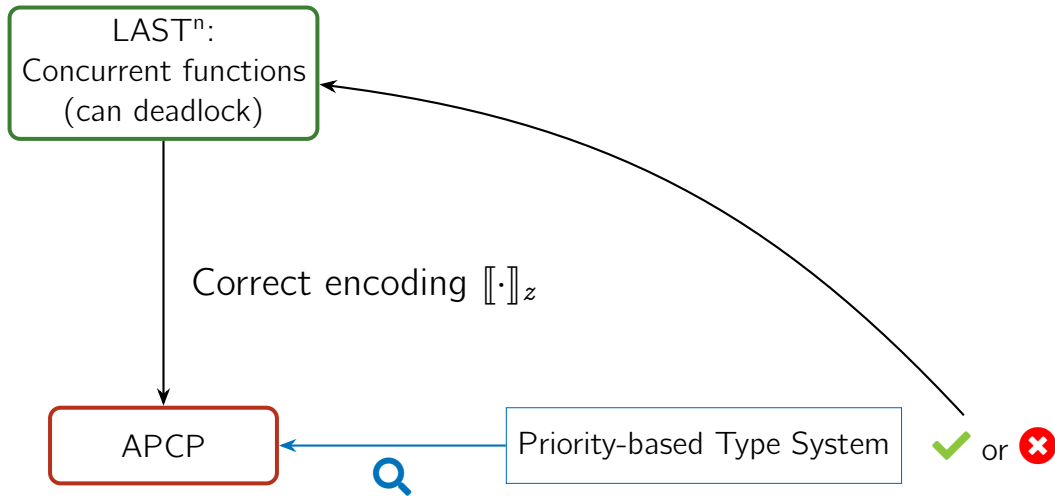
Application: DF in Concurrent Functional Sessions



Application: DF in Concurrent Functional Sessions



Application: DF in Concurrent Functional Sessions



Summing Up

Three typed asynchronous π -calculi based on Linear Logic:

1. AP: processes correctly follow protocols but may deadlock
2. ACP: AP with logic-based composition, DF for tree-like topologies
3. APCP: extension of AP with priorities, DF for circular topologies

Summing Up

Three typed asynchronous π -calculi based on Linear Logic:

1. AP: processes correctly follow protocols but may deadlock
2. ACP: AP with logic-based composition, DF for tree-like topologies
3. APCP: extension of AP with priorities, DF for circular topologies

Further Results

- ▶ Processes with recursion and recursive session types (tail-recursive)
- ▶ DF for LAST^n (concurrent functional sessions) via an encoding into APCP
- ▶ Analysis of multiparty protocols with APCP

Summing Up

Three typed asynchronous π -calculi based on Linear Logic:

1. AP: processes correctly follow protocols but may deadlock
2. ACP: AP with logic-based composition, DF for tree-like topologies
3. APCP: extension of AP with priorities, DF for circular topologies

Further Results

- ▶ Processes with recursion and recursive session types (tail-recursive)
- ▶ DF for LAST^n (concurrent functional sessions) via an encoding into APCP
- ▶ Analysis of multiparty protocols with APCP

Summing Up

Three typed asynchronous π -calculi based on Linear Logic:

1. AP: processes correctly follow protocols but may deadlock
2. ACP: AP with logic-based composition, DF for tree-like topologies
3. APCP: extension of AP with priorities, DF for circular topologies

A Recent Result (FORTE'26, with Andreia Mordido)

- ▶ Priority-based DF in a concurrent functional setting: the **FreeST language**.
- ▶ **Context-free** session types: non-regular protocols and tree-like structures. Polymorphic recursion at work.
- ▶ Prior type systems admit deadlocked programs as typable. Ours is the first that enforces DF.
- ▶ Handling priorities in programs with polymorphic sessions with recursion is technically subtle.

Unrestricted Behavior: Clients and Servers

Keywords

Concurrency Theory, Message-Passing, Programming Languages, Verification

Keywords

Concurrency Theory, Message-Passing, Programming Languages, Verification

- ▶ *Type systems*

Well-typed programs can't go wrong (Milner)

- ▶ *Session types* for communication correctness

What and **when** should be sent through a channel

- ▶ *Process calculi*

The π -calculus treats **processes** like the λ -calculus treats **functions**

Keywords

Concurrency Theory, Message-Passing, Programming Languages, Verification

- ▶ *Type systems*

Well-typed programs can't go wrong (Milner)

- ▶ *Session types* for communication correctness

What and **when** should be sent through a channel

- ▶ *Process calculi*

The π -calculus treats **processes** like the λ -calculus treats **functions**

- ▶ *Propositions as sessions*

Tight connection between **logic** and **type theory**.

Propositions As Types

Intuitionistic logic propositions	$\leftrightarrow$	types describing data
Natural deduction derivations	$\leftrightarrow$	λ -calculus terms
Proof normalization reductions	$\leftrightarrow$	β -reductions

aka Curry-Howard correspondence, formulae-as-types, proofs-as-programs...

Propositions As Sessions

Linear	logic propositions	$\leftrightarrow$	types describing behavior (sessions)
Sequence calculus	derivations	$\leftrightarrow$	π -calculus processes
Cut	reductions	$\leftrightarrow$	communication between processes

Propositions As Sessions

Linear	logic propositions	$\leftrightarrow$	types describing behavior (sessions)
Sequence calculus	derivations	$\leftrightarrow$	π -calculus processes
Cut	reductions	$\leftrightarrow$	communication between processes

Today: incorporating *servers* into the framework.

Session types for MAILL

....

n.b. we write $\bar{x}(y).P$ for $(\nu y)\bar{x}\langle y\rangle.P$

Outline

Servers and the ! Modality

Linear Logic Menu

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (€3 for
a glass)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \& Soup) \otimes \\ ((Schn \& Tofu \& (E \otimes E \multimap Fish)) \\ \otimes Fries) \otimes \\ (Icecr \& Cheese) \otimes \\ (E^3 \multimap Beer) \end{array} \right)$$

Linear Logic Menu

Summer Menu! €15 p.p.

Starter: Greek Salad, or
Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for
additional €2)

(all main dishes come with a
side of fries)

Desert: Ice Cream, or
Cheese Platter

Drinks: Leffe Tripel (€3 for
a glass)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \& Soup) \otimes \\ ((Schn \& Tofu \& (E \otimes E \multimap Fish)) \\ \otimes Fries) \otimes \\ (Icecr \& Cheese) \otimes \\ (E^3 \multimap Beer) \end{array} \right)$$

Restricted vs unrestricted resources

Intuitionistic (and classical) logic:

$$A \wedge (A \rightarrow B) \vdash B \wedge A \wedge (A \rightarrow B)$$

Linear Logic:

$$A \otimes (A \multimap B) \vdash B$$

$$A \otimes (A \multimap B) \vdash A \otimes (A \multimap B)$$

Restricted vs unrestricted resources

Intuitionistic (and classical) logic:

$$A \wedge (A \rightarrow B) \vdash B \wedge A \wedge (A \rightarrow B)$$

Linear Logic:

$$A \otimes (A \multimap B) \vdash B$$

$$A \otimes (A \multimap B) \vdash A \otimes (A \multimap B)$$

$$A \otimes (A \multimap B) \vdash B \& (A \otimes (A \multimap B))$$

Unrestricted Resources

- ▶ So far we have talked only about *linear* resources

Unrestricted Resources

- ▶ So far we have talked only about *linear* resources
- ▶ A session type A describes a finite, linear session

Unrestricted Resources

- ▶ So far we have talked only about *linear* resources
- ▶ A session type A describes a finite, linear session
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

Unrestricted Resources

- ▶ So far we have talked only about *linear* resources
- ▶ A session type A describes a finite, linear session
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$



Divide all the resources between P and Q

Unrestricted Resources

- ▶ So far we have talked only about *linear* resources
- ▶ A session type A describes a finite, linear session
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

- ▶ Question: how to introduce *unrestricted*/non-linear sessions?

The ! Modality

$!A$ *of course A, or bang A, or exponential A*

$$!A \vdash A$$

$$!A \vdash !A \otimes !A$$

$$!A \vdash 1$$

$$\frac{A \vdash B}{!A \vdash !B}$$

- ▶ $!A$ satisfies contraction, weakening

The ! Modality

$!A$ *of course A, or bang A, or exponential A*

$$\begin{array}{cccc} !A \vdash A & !A \vdash !A \otimes !A & !A \vdash 1 & \frac{A \vdash B}{!A \vdash !B} \end{array}$$

- ▶ $!A$ satisfies contraction, weakening
- ▶ list-like presentation: $!A \vdash 1 \ \& \ (A \otimes !A)$

The ! Modality

$!A$ *of course A, or bang A, or exponential A*

$$\begin{array}{cccc} !A \vdash A & !A \vdash !A \otimes !A & !A \vdash 1 & \frac{A \vdash B}{!A \vdash !B} \end{array}$$

- ▶ $!A$ satisfies contraction, weakening
- ▶ list-like presentation: $!A \vdash 1 \ \& \ (A \otimes !A)$
- ▶ $A \otimes !(A \multimap B) \vdash B \otimes !(A \multimap B)$

The ! Modality

Some useful properties of !:

► $!(A \& B) \dashv\vdash !A \otimes !B$

The ! Modality

Some useful properties of !:

▶ $!(A \& B) \dashv\vdash !A \otimes !B$

▶ $!(A \otimes B) \dashv\vdash !A \otimes !B$

The ! Modality

Some useful properties of !:

- ▶ $!(A \& B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \otimes B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \& B) \vdash !A \& !B$
- ▶ $!A \& !B \not\vdash !(A \& B)$

The ! Modality

Some useful properties of !:

- ▶ $!(A \& B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \otimes B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \& B) \vdash !A \& !B$
- ▶ $!A \& !B \not\vdash !(A \& B)$
- ▶ $!A \dashv\vdash !!A$

The ! Modality as a Session Type

- ▶ $!A$ - unlimited copies of the session A / server for session A

The ! Modality as a Session Type

- ▶ $!A$ - unlimited copies of the session A / server for session A
- ▶ We have two kinds of contexts: $\Gamma; \Delta \vdash C$
 - ▶ DILL - Dual Intuitionistic Linear Logic
 - ▶ Unrestricted resources in Γ , restricted resources in Δ
 - ▶ $A_1, \dots, A_k; B_1, \dots, B_n \vdash C$ stands for $!A_1 \otimes \dots \otimes !A_k \otimes B_1 \otimes \dots \otimes B_n \multimap C$

The ! Modality as a Session Type

- ▶ $!A$ - unlimited copies of the session A / server for session A
- ▶ We have two kinds of contexts: $\Gamma; \Delta \vdash C$
 - ▶ DILL - Dual Intuitionistic Linear Logic
 - ▶ Unrestricted resources in Γ , restricted resources in Δ
 - ▶ $A_1, \dots, A_k; B_1, \dots, B_n \vdash C$ stands for $!A_1 \otimes \dots \otimes !A_k \otimes B_1 \otimes \dots \otimes B_n \multimap C$
- ▶ Unrestricted resources are non-linear and can be shared:

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

The ! Modality as a Session Type

- ▶ $!A$ - unlimited copies of the session A / server for session A
- ▶ We have two kinds of contexts: $\Gamma; \Delta \vdash C$
 - ▶ DILL - Dual Intuitionistic Linear Logic
 - ▶ Unrestricted resources in Γ , restricted resources in Δ
 - ▶ $A_1, \dots, A_k; B_1, \dots, B_n \vdash C$ stands for $!A_1 \otimes \dots \otimes !A_k \otimes B_1 \otimes \dots \otimes B_n \multimap C$
- ▶ Unrestricted resources are non-linear and can be shared:

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

The ! Modality as a Session Type

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: y : A \quad \Gamma; \Delta_2 \vdash Q :: x : B}{\Gamma; \Delta_1, \Delta_2 \vdash \bar{x}\langle y \rangle.(P \mid Q) :: x : A \otimes B} \quad \Gamma; x : A \vdash [y \leftarrow x] :: y :: A \quad \dots etc$$

The ! Modality as a Session Type

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: y : A \quad \Gamma; \Delta_2 \vdash Q :: x : B}{\Gamma; \Delta_1, \Delta_2 \vdash \bar{x}\langle y \rangle.(P \mid Q) :: x : A \otimes B}$$

$$\Gamma; x : A \vdash [y \leftarrow x] :: y :: A \quad \dots etc$$

$$\frac{x : A, y : A, \Gamma; \Delta \vdash P :: z : C}{x : A, \Gamma; \Delta \vdash P\{x/y\} :: z : C}$$

$$\frac{\Gamma; \Delta \vdash P :: z : C}{x : A, \Gamma; \Delta \vdash P :: z : C}$$

The ! Modality: Servers

$$\frac{\Gamma; \emptyset \vdash \quad A}{\Gamma; \emptyset \vdash !A}$$

- ▶ Can derive $!A$ from A , if the proof of A does **not** depend on restricted resources

The ! Modality: Servers

$$\frac{\Gamma; \emptyset \vdash P :: y : A}{\Gamma; \emptyset \vdash !u(y).P :: u : !A}$$

- ▶ Can derive $!A$ from A , if the proof of A does **not** depend on restricted resources
- ▶ $!u(y).P$ receives a request (with a name) on u , and provide P for that request

The ! Modality: Clients

$$\frac{\Gamma; x : !A, \Delta \vdash}{u : A, \Gamma; \Delta \vdash} C$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash}{u : A, \Gamma; \Delta \vdash} C$$

The ! Modality: Clients

$$\frac{\Gamma; x : !A, \Delta \vdash P :: z : C}{u : A, \Gamma; \Delta \vdash P\{u/x\} :: z : C}$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash C}{u : A, \Gamma; \Delta \vdash C}$$

- ▶ Can move $!A$ from the restricted to the unrestricted section

The ! Modality: Clients

$$\frac{\Gamma; x : !A, \Delta \vdash P :: z : C}{u : A, \Gamma; \Delta \vdash P\{u/x\} :: z : C}$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash Q :: z : C}{u : A, \Gamma; \Delta \vdash (\nu y)(\overline{u}\langle y \rangle.Q) :: z : C}$$

- ▶ Can move $!A$ from the restricted to the unrestricted section
- ▶ Client requests a copy of A by creating a new name and sending the request to $!A$

The ! Modality: Client-Server interaction

$$\frac{
 \frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad
 \frac{
 \frac{\Gamma, u : A; \Delta, x : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu x)(\bar{u}\langle x \rangle.Q)}
 }{
 \Gamma; \Delta, u : !A \vdash (\nu x)(\bar{u}\langle x \rangle.Q)
 }
 }{
 \Gamma; \Delta, u : !A \vdash (\nu u)(!u(x).P \mid (\nu x)(\bar{u}\langle x \rangle.Q))
 }
 \longrightarrow$$

The ! Modality: Client-Server interaction

$$\begin{array}{c}
 \frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad \frac{\frac{\Gamma, u : A; \Delta, x : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu x)(\bar{u}\langle x \rangle.Q)} \quad \Gamma; \Delta, u : !A \vdash (\nu x)(\bar{u}\langle x \rangle.Q)}{\Gamma; \Delta, u : !A \vdash (\nu u)(!u(x).P \mid (\nu x)(\bar{u}\langle x \rangle.Q))} \\
 \longrightarrow \\
 \frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma, u : A; \emptyset \vdash P :: x : A} \quad \Gamma, u : A; \Delta, x : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu x)(P \mid Q) :: z : C}
 \end{array}$$

The ! Modality: Client-Server interaction

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad \frac{\frac{\Gamma, u : A; \Delta, x : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu x)(\bar{u}\langle x \rangle.Q)} \quad \Gamma; \Delta, u : !A \vdash (\nu x)(\bar{u}\langle x \rangle.Q)}{\Gamma; \Delta, u : !A \vdash (\nu u)(!u(x).P \mid (\nu x)(\bar{u}\langle x \rangle.Q))}$$

→

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma, u : A; \emptyset \vdash P :: x : A} \quad \frac{\Gamma, u : A; \Delta, x : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu x)(P \mid Q) :: z : C} \quad \Gamma; \emptyset \vdash !u(x).P :: u : !A}{\Gamma; \Delta \vdash (\nu u)(!u(x).P \mid (\nu x)(P \mid Q)) :: z : C}$$

Client-Server interaction

$$(\nu u)(!u(y).P \mid (\nu y)(\overline{u}\langle y \rangle \mid Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

Client-Server interaction

$$(\nu u)(!u(y).P \mid (\nu y)(\bar{u}\langle y \rangle \mid Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

For convenience, we will also use the derived composition rule for servers

$$\frac{\begin{array}{c} \text{!-cut} \\ \Gamma; \Delta_1 \vdash P :: x : !A \quad \Gamma, x : A; \Delta_2 \vdash Q :: z : C \end{array}}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

The π DILL system

$$\frac{!R \quad \Gamma; \emptyset \vdash P :: y : A}{\Gamma; \emptyset \vdash !u(y).P :: u : !A}$$

$$\frac{!L \quad u : A, \Gamma; y : A, \Delta \vdash Q :: z : C}{u : A, \Gamma; \Delta \vdash (\nu y)(\bar{u}\langle y \rangle.Q) :: z : C}$$

$$\frac{!-move \quad \Gamma; \Delta, x : !A \vdash P :: z : C}{u : A, \Gamma; \Delta \vdash P\{u/x\} :: z : C}$$

$$\frac{!-cut \quad \Gamma; \Delta_1 \vdash P :: x : !A \quad \Gamma, x : A; \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

$$(\nu u)(!u(y).P \mid (\nu y)(\bar{u}\langle y \rangle \mid Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

List-like specification for Servers

$$\frac{u : A; \emptyset \vdash \dots :: w : 1 \& (A \otimes !A)}{\emptyset; u : !A \vdash \dots :: w : 1 \& (A \otimes !A)}$$

List-like specification for Servers

$$\begin{array}{c}
 \frac{u : A; \emptyset \vdash \dots :: w : 1}{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)} \quad \frac{}{u : A; \emptyset \vdash \dots :: w : A \otimes !A} \\
 \hline
 \frac{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}{\emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}
 \end{array}$$

List-like specification for Servers

$$\begin{array}{c}
 \frac{u : A; \emptyset \vdash \overline{w}\langle \rangle :: w : 1 \quad \frac{}{u : A; \emptyset \vdash \dots :: w : A \otimes !A}}{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)} \\
 \frac{}{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)} \\
 \frac{}{\emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}
 \end{array}$$

List-like specification for Servers

$$\begin{array}{c}
 u : A; \emptyset \vdash \overline{u}(x).[y \leftarrow x] :: y : A \\
 u : A; \emptyset \vdash [w \leftarrow u] :: w :: !A \\
 \hline
 u : A; \emptyset \vdash \overline{w}\langle \rangle :: w : 1 \quad u : A; \emptyset \vdash \overline{w}(y).(\overline{u}(x).[y \leftarrow x]) \mid [w \leftarrow u] :: w : A \otimes !A \\
 \hline
 u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A) \\
 \hline
 u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A) \\
 \hline
 \emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)
 \end{array}$$

Bar Tending at the Linear Logic Cafe

$$\begin{array}{c}
 \frac{\emptyset; e : E \otimes E \otimes E \vdash \text{deposit}(e, x) :: x : 1 \quad \frac{\frac{\emptyset; k : \text{Keg} \vdash \text{pour}(k, b) :: b : \text{Beer}}{u : \text{Keg}; \emptyset \vdash \bar{u}(k).\text{pour}(k, b) :: b : \text{Beer}}}{u : \text{Keg}; x : 1 \vdash x().\bar{u}(k).\text{pour}(k, b) :: b : \text{Beer}}}{u : \text{Keg}; e : E^3 \vdash (\nu x)(\text{deposit}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: b : \text{Beer}} \\
 \frac{u : \text{Keg}; \emptyset \vdash b(e).(\nu x)(\text{deposit}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: b : E^3 \multimap \text{Beer}}{u : \text{Keg}; \emptyset \vdash !t(b).b(e).(\nu x)(\text{deposit}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: t : !(E^3 \multimap \text{Beer})}
 \end{array}$$

Bunched Implications

The logic of Bunched Implications

So far we have seen π DILL based on intuitionistic linear logic.

- ▶ A formula signifies amount of resources.
- ▶ $A \otimes B$: one copy of A , plus one copy of B .
- ▶ Models *quantity* of resources
- ▶ Sharing is allowed through the $!$ modality.

In this lecture we will talk about an alternative logic for resources: BI and π BI.

The logic of Bunched Implications

$$A, B ::= 1 \mid A * B \mid A \multimap B \mid \top \mid A \wedge B \mid A \rightarrow B \mid A \vee B$$

- ▶ A formula signifies resources owned.
- ▶ $A * B$: own resources can be separated into resources denoted by A , and resources denoted by B .
- ▶ Models *ownership* (and separation) of resources.
- ▶ Sharing is allowed through intuitionistic connectives
 - ▶ $A \wedge B$: own resources satisfy both A and B

NB: Separation Logic

BI forms a basis for *separation logic*...

- ▶ $\ell \mapsto v$: the current state has the location ℓ in memory, and it stores the value v
- ▶ $P * Q$: the current state can be divided into two disjoint parts, for which P and Q hold respectively

NB: Separation Logic

BI forms a basis for *separation logic*...

- ▶ $\ell \mapsto v$: the current state has the location ℓ in memory, and it stores the value v
- ▶ $P * Q$: the current state can be divided into two disjoint parts, for which P and Q hold respectively
- ▶ $\ell \mapsto v * \ell' \mapsto v'$: the locations ℓ and ℓ' do not alias each other
- ▶ $\ell \mapsto v \wedge \ell' \mapsto v'$: aliasing is allowed

NB: Separation Logic

BI forms a basis for *separation logic*...

- ▶ $\ell \mapsto v$: the current state has the location ℓ in memory, and it stores the value v
- ▶ $P * Q$: the current state can be divided into two disjoint parts, for which P and Q hold respectively
- ▶ $\ell \mapsto v * \ell' \mapsto v'$: the locations ℓ and ℓ' do not alias each other
- ▶ $\ell \mapsto v \wedge \ell' \mapsto v'$: aliasing is allowed
- ▶ $\ell_1 \mapsto (v_1, \ell_2) * \ell_2 \mapsto (v_2, \ell_3) * \dots * \ell_n \mapsto (v_n, \ell_o) * \ell_o \mapsto \text{NULL}$:
a linked list without cycles

Outline

BI Proof Theory

Bunches in BI

Contexts in linear logic can be formalized as a data type:

$$\begin{aligned}\Delta &::= \emptyset \mid A, \Delta && \text{or} \\ \Delta &::= \emptyset \mid A \mid \Delta_1, \Delta_2\end{aligned}$$

The composition Δ_1, Δ_2 externalizes the $\otimes$ connective (c.f. left and right rules for $\otimes$). The $\&$ operator by contrast did not an “external” version.

Bunches in BI

Contexts in linear logic can be formalized as a data type:

$$\begin{aligned}\Delta &::= \emptyset \mid A, \Delta && \text{or} \\ \Delta &::= \emptyset \mid A \mid \Delta_1, \Delta_2\end{aligned}$$

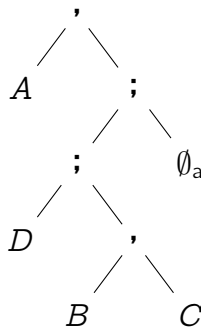
The composition Δ_1, Δ_2 externalizes the $\otimes$ connective (c.f. left and right rules for $\otimes$). The $\&$ operator by contrast did not an “external” version.

In BI we externalize both multiplicative $*$ and additive $\wedge$ on equal footing.

Bunches in BI

Bunches are trees of formulas with two kinds of nodes:

$$\Delta ::= A \mid \emptyset_a \mid \emptyset_m \mid \Delta_1 ; \Delta_2 \mid \Delta_1 , \Delta_2$$



For example: $A, (D; (B, C); \emptyset_a)$.

Bunch equivalence

$\equiv$ is the smallest congruence generated by

$$\Delta_1 , \Delta_2 \equiv \Delta_2 , \Delta_1$$

$$\Delta , \emptyset_m \equiv \Delta$$

$$\Delta_1 , (\Delta_2 , \Delta_3) \equiv (\Delta_1 , \Delta_2) , \Delta_3$$

$$\Delta_1 ; \Delta_2 \equiv \Delta_2 ; \Delta_1$$

$$\Delta ; \emptyset_a \equiv \Delta$$

$$\Delta_1 ; (\Delta_2 ; \Delta_3) \equiv (\Delta_1 ; \Delta_2) ; \Delta_3$$

E.g. $A , (D ; (B , C) ; \emptyset_a) \equiv ((B , C) ; D) , A$

Bunch equivalence

$\equiv$ is the smallest congruence generated by

$$\Delta_1 , \Delta_2 \equiv \Delta_2 , \Delta_1$$

$$\Delta , \emptyset_m \equiv \Delta$$

$$\Delta_1 , (\Delta_2 , \Delta_3) \equiv (\Delta_1 , \Delta_2) , \Delta_3$$

$$\Delta_1 ; \Delta_2 \equiv \Delta_2 ; \Delta_1$$

$$\Delta ; \emptyset_a \equiv \Delta$$

$$\Delta_1 ; (\Delta_2 ; \Delta_3) \equiv (\Delta_1 ; \Delta_2) ; \Delta_3$$

E.g. $A , (D ; (B , C) ; \emptyset_a) \equiv ((B , C) ; D) , A$

We will work with bunches modulo $\equiv$

BI sequent calculus

Sequent: $\Gamma \vdash A$

$$\frac{\Gamma; A; B \vdash C}{\Gamma; A \wedge B \vdash C}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1; \Gamma_2 \vdash A \wedge B}$$

BI sequent calculus

Left and right rules


$$\frac{\Gamma; A; B \vdash C}{\Gamma; A \wedge B \vdash C}$$

$$\frac{\Gamma; \Gamma \vdash C}{\Gamma \vdash C}$$


$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1; \Gamma_2 \vdash A \wedge B}$$

$$\frac{\Gamma \vdash C}{\Gamma; \Gamma' \vdash C}$$

BI sequent calculus

Structural rules

$$\frac{\Gamma; A; B \vdash C}{\Gamma; A \wedge B \vdash C}$$

$$\frac{\Gamma; \Gamma \vdash C}{\Gamma \vdash C}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1; \Gamma_2 \vdash A \wedge B}$$

$$\frac{\Gamma \vdash C}{\Gamma; \Gamma' \vdash C}$$

BI sequent calculus

$$\frac{\Gamma ; (A, B) \vdash C}{\Gamma ; A * B \vdash C}$$

$$\frac{\Gamma ; A ; B \vdash C}{\Gamma ; A \wedge B \vdash C}$$

$$\frac{\Gamma ; \Gamma \vdash C}{\Gamma \vdash C}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A * B}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1 ; \Gamma_2 \vdash A \wedge B}$$

$$\frac{\Gamma \vdash C}{\Gamma ; \Gamma' \vdash C}$$

BI sequent calculus

$$\frac{\Delta(A, B) \vdash C}{\Delta(A * B) \vdash C}$$

$$\frac{\Delta(A ; B) \vdash C}{\Delta(A \wedge B) \vdash C}$$

$$\frac{\Delta(\Gamma ; \Gamma) \vdash C}{\Delta(\Gamma) \vdash C}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A * B}$$

$$\frac{\Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1 ; \Gamma_2 \vdash A \wedge B}$$

$$\frac{\Delta(\Gamma) \vdash C}{\Delta(\Gamma ; \Gamma') \vdash C}$$

$\Delta(-)$ is a bunch with a hole, $\Delta(\Gamma)$ plugs Γ into the hole

BI sequent calculus

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta; A \vdash B}{\Delta \vdash A \rightarrow B}$$

BI sequent calculus

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta ; A \vdash B}{\Delta \vdash A \rightarrow B}$$

- Sequent calculus for BI externalizes $\wedge$ and $*$ as different connectives: $;$ and $,$. Only $;$ admits weakening and contraction.

BI sequent calculus

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta ; A \vdash B}{\Delta \vdash A \rightarrow B}$$

- ▶ Sequent calculus for BI externalizes $\wedge$ and $*$ as different connectives: $;$ and $,$. Only $;$ admits weakening and contraction.
- ▶ Because of that, contexts in the sequents are not lists/multisets, but *bunches*;

BI sequent calculus

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta ; A \vdash B}{\Delta \vdash A \rightarrow B}$$

- ▶ Sequent calculus for BI externalizes $\wedge$ and $*$ as different connectives: $;$ and $,$. Only $;$ admits weakening and contraction.
- ▶ Because of that, contexts in the sequents are not lists/multisets, but *bunches*;
- ▶ Left rules can be applied deep inside an arbitrary *bunched context*.

BI sequent calculus

Structural rules can be applied deep inside a bunched context, including the cut rule:

$$\frac{\Gamma \vdash A \quad \Delta(A) \vdash C}{\Delta(\Gamma) \vdash C}$$

BI vs Linear Logic

- ▶ As it stands, BI and ILL are incompatible
- ▶ BI is conservative over MILL and Intuitionistic Logic

BI vs Linear Logic

- ▶ As it stands, BI and ILL are incompatible
- ▶ BI is conservative over MILL and Intuitionistic Logic
 - ▶ $A \otimes B \otimes (C \multimap D)$ is provable in MILL iff $A * B * (C \multimap^* D)$ is provable in BI

BI vs Linear Logic

- ▶ As it stands, BI and ILL are incompatible
- ▶ BI is conservative over MILL and Intuitionistic Logic
 - ▶ $A \otimes B \otimes (C \multimap D)$ is provable in MILL iff $A * B * (C \multimap D)$ is provable in BI
- ▶ BI is not conservative over MAILL
 - ▶ Distributivity of $\wedge$ over $\vee$ holds in BI: $A \wedge (B \vee C) \vdash (A \wedge B) \vee (A \wedge C)$
 - ▶ But not in ILL: $A \& (B \oplus C) \not\vdash (A \& B) \oplus (A \& C)$

π BI calculus

We would like to have a session-types interpretation of BI that is conservative over the MILL fragment of π DILL.

- ▶ We are “forced to interpret” $*$ as output, $\multimap$ as input.
- ▶ In fact, we will also treat $\wedge$ as output and $\rightarrow$ as input.
- ▶ The main difference will be in treatment of structural rules for $\wedge$.

π BI calculus: additives and multiplicatives

$$\frac{\Pi(y : A, x : B) \vdash P :: z : C}{\Pi(x : A * B) \vdash x(y).P :: z : C}$$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash \overline{x}(y).(P \mid Q) :: x : A * B}$$

π BI calculus: additives and multiplicatives

$$\frac{\Pi(y : A, x : B) \vdash P :: z : C}{\Pi(x : A * B) \vdash x(y).P :: z : C}$$

$$\frac{\Pi(y : A ; x : B) \vdash P :: z : C}{\Pi(x : A \wedge B) \vdash x(y).P :: z : C}$$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1, \Delta_2 \vdash \overline{x}(y).(P \mid Q) :: x : A * B}$$

$$\frac{\Delta_1 \vdash P :: y : A \quad \Delta_2 \vdash Q :: x : B}{\Delta_1 ; \Delta_2 \vdash \overline{x}(y).(P \mid Q) :: x : A \wedge B}$$

π BI calculus: additives and multiplicatives

$\rightarrow$ vs $\multimap$ and 1_m and 1_a .

π BI calculus: explicit structural rules

$$\frac{\Pi(x_1 : A ; x_2 : A) \vdash P :: z : C}{\Pi(x : A) \vdash P :: z : C}$$

$$\frac{\Pi(\emptyset_a) \vdash P :: z : C}{\Pi(x : A) \vdash P :: z : C}$$

π BI calculus: explicit structural rules

$$\frac{\Pi(x_1 : A ; x_2 : A) \vdash P :: z : C}{\Pi(x : A) \vdash \rho[x \mapsto x_1, x_2].P :: z : C}$$

$$\frac{\Pi(\emptyset_a) \vdash P :: z : C}{\Pi(x : A) \vdash \rho[x \mapsto \emptyset].P :: z : C}$$

Spawn

Spawn construct:

- ▶ $\rho[x \mapsto y, z].P$: spawn two copies of processes on x , bind them to y, z , and proceed as P
- ▶ $\rho[x \mapsto \emptyset].P$: kill the process on x , and proceed as P
- ▶ General form: $\rho[\sigma].P$ where $\sigma : Name \xrightarrow{\text{fin}} \wp(Name)$
 - ▶ $\forall x, y \in \text{dom}(\sigma). x \neq y \implies \sigma(x) \cap \sigma(y) = \emptyset$
 - ▶ $\forall x \in \text{dom}(\sigma). \sigma(x) \cap \text{dom}(\sigma) = \emptyset$

π BI calculus: explicit structural rules

$$\frac{\Pi(\Delta^{(1)} ; \Delta^{(2)}) \vdash P :: z : C}{\Pi(\Delta) \vdash \rho[x \mapsto x_1, x_2 \mid x \in \Delta].P :: z : C}$$

$$\frac{\Pi(\emptyset_a) \vdash P :: z : C}{\Pi(\Delta) \vdash \rho[x \mapsto \emptyset \mid x \in \Delta].P :: z : C}$$

Spawn: reductions

$$\frac{\emptyset_a \vdash P :: x : A \quad \frac{\Pi(x_1 : A ; x_2 : A) \vdash Q :: z : C}{\Pi(x : A) \vdash \rho[x \mapsto x_1, x_2].Q :: z : C}}{\Pi(\emptyset_a) \vdash (\nu x)(P \mid \rho[x \mapsto x_1, x_2].Q) :: z : C}$$

$$\frac{\emptyset_a \vdash P[x_2/x] :: x_2 : A \quad \frac{\emptyset_a \vdash P[x_1/x] :: x_1 : A \quad \Pi(x_1 : A ; x_2 : A) \vdash Q :: z : C}{\Pi(\emptyset_a ; x_2 : A) \vdash (\nu x_1)(P\{x_1/x\} \mid Q) :: z : C}}{\Pi(\emptyset_a ; \emptyset_a) \vdash (\nu x_2)(P\{x_2/x\} \mid (\nu x_1)(P\{x_1/x\} \mid Q)) :: z : C}$$

Spawn: reductions

$$(\nu x)(P \mid_x \rho[x \mapsto x_1, x_2]. Q) \longrightarrow (\nu x_2)(P\{x_2/x\} \mid_{x_2} (\nu x_1)(P\{x_1/x\} \mid_{x_1} Q))$$

Spawn: reductions

$$\frac{\Delta \vdash P :: x : A \quad \frac{\Pi(x_1 : A ; x_2 : A) \vdash Q :: z : C}{\Pi(x : A) \vdash \rho[x \mapsto x_1, x_2].Q :: z : C}}{\Pi(\Delta) \vdash (\nu x)(P \mid \rho[x \mapsto x_1, x_2].Q) :: z : C}$$

$$\frac{\Delta^{(2)} \vdash P\{x_2/x\} :: x_2 : A \quad \frac{\Delta^{(1)} \vdash P\{x_1/x\} :: x_1 : A \quad \Pi(x_1 : A ; x_2 : A) \vdash Q :: z : C}{\Pi(\Delta^{(1)} ; x_2 : A) \vdash (\nu x_1)(P\{x_1/x\} \mid Q) :: z : C}}{\Pi(\Delta^{(1)} ; \Delta^{(2)}) \vdash (\nu x_2)(P\{x_2/x\} \mid (\nu x_1)(P\{x_1/x\} \mid Q)) :: z : C}$$

Spawn: reductions

$$(\nu x)(P \mid_x \rho[x \mapsto x_1, x_2]. Q)$$

$$\longrightarrow$$

$$\rho[y \mapsto y_1, y_2 \mid y \in \text{fn}(P) \setminus \{x\}]. (\nu x_2)(P^{(2)} \mid_{x_2} (\nu x_1)(P^{(1)} \mid_{x_1} Q))$$

Spawn: structural congruence

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{y : A \vdash \rho[y \mapsto y_1, y_2, y_3].P :: u : C}}{x : B ; y : A \vdash \rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P :: u : C}$$

Spawn: structural congruence

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{y : A \vdash \rho[y \mapsto y_1, y_2, y_3].P :: u : C}}{x : B ; y : A \vdash \rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P :: u : C}$$

$\equiv$

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{x : B ; y_1 : A ; y_2 : A ; y_3 : A \vdash \rho[x \mapsto \emptyset].P :: u : C}}{x : B ; y : A \vdash \rho[y \mapsto y_1, y_2, y_3].\rho[x \mapsto \emptyset].P :: u : C}$$

$$\begin{aligned}\rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P &\equiv \rho[y \mapsto y_1, y_2, y_3].\rho[x \mapsto \emptyset].P \\ \rho[\sigma_1].\rho[\sigma_2].P &\equiv \rho[\sigma_2].\rho[\sigma_1].P\end{aligned}$$

Spawn: merge

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{y : A \vdash \rho[y \mapsto y_1, y_2, y_3].P :: u : C}}{x : B ; y : A \vdash \rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P :: u : C}$$

Spawn: merge

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{y : A \vdash \rho[y \mapsto y_1, y_2, y_3].P :: u : C}}{x : B ; y : A \vdash \rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P :: u : C}$$

$\longrightarrow$

$$\frac{y_1 : A ; y_2 : A ; y_3 \vdash P :: u : C}{x : B ; y : A \vdash \rho[x \mapsto \emptyset, y \mapsto y_1, y_2, y_3].P :: u : C}$$

$$\rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P \longrightarrow \rho[x \mapsto \emptyset, y \mapsto y_1, y_2, y_3].P$$

Spawn: merge

$$\frac{\frac{y_1 : A ; y_2 : A ; y_3 : A \vdash P :: u : C}{y : A \vdash \rho[y \mapsto y_1, y_2, y_3].P :: u : C}}{x : B ; y : A \vdash \rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P :: u : C}$$

$\longrightarrow$

$$\frac{y_1 : A ; y_2 : A ; y_3 \vdash P :: u : C}{x : B ; y : A \vdash \rho[x \mapsto \emptyset, y \mapsto y_1, y_2, y_3].P :: u : C}$$

$$\rho[x \mapsto \emptyset].\rho[y \mapsto y_1, y_2, y_3].P \longrightarrow \rho[x \mapsto \emptyset, y \mapsto y_1, y_2, y_3].P$$

$$\rho[\sigma_1].\rho[\sigma_2].P \longrightarrow \rho[\sigma_1 \mathrel{\wedge\!\!\wedge} \sigma_2].P$$

Spawn: merge

$$\left[\begin{array}{l} x \mapsto \emptyset \\ y \mapsto \{y_1, y_2, y_3\} \end{array} \right] \mathbin{\wedge\!\wedge} \left[\begin{array}{l} y_2 \mapsto \emptyset \\ y_3 \mapsto \{y_4, y_5\} \\ z \mapsto z_1 \end{array} \right] = \left[\begin{array}{l} x \mapsto \emptyset \\ y \mapsto \{y_1, y_4, y_5\} \\ z \mapsto z_1 \end{array} \right]$$

Spawn: merge

$$(\sigma_1 \mathbb{M} \sigma_2)(x) = \begin{cases} \sigma_2[\sigma_1(x)] \cup (\sigma_1(x) \setminus \text{dom}(\sigma_2)) & x \in \text{dom}(\sigma_1) \\ \sigma_2(x) & x \notin \text{dom}(\sigma_1) \wedge x \notin \text{im}(\sigma_1) \\ \perp & \text{otherwise} \end{cases}$$

Spawn typing

With merge we can get spawns $\rho[\sigma \mapsto \cdot]P$ to go beyond just weakening/contraction.
To type those intermediate spawns we have $\sigma : \Delta_1 \rightsquigarrow \Delta_2$

$$[x \mapsto \{x_1, \dots, x_n\} \mid x \in \Delta] : \Pi(\Delta) \rightsquigarrow \Pi(\Delta^{(1)} ; \dots ; \Delta^{(n)})$$

$$[x \mapsto \emptyset \mid x \in \Delta_1] : \Pi(\Delta_1 ; \Delta_2) \rightsquigarrow \Pi(\Delta_2) \qquad \frac{\sigma_1 : \Delta_0 \rightsquigarrow \Delta_1 \quad \sigma_2 : \Delta_1 \rightsquigarrow \Delta_2}{(\sigma_1 \mathbin{\mathbb{M}} \sigma_2) : \Delta_0 \rightsquigarrow \Delta_2}$$

$$[x \mapsto \emptyset, y \mapsto \{y_1, y_2, y_3\}] : \Pi(x : B ; y : A) \rightsquigarrow \Pi(y_1 : A ; y_2 : A ; y_3 : A)$$

Spawn typing

With merge we can get spawns $\rho[\sigma \mapsto \cdot]P$ to go beyond just weakening/contraction.
To type those intermediate spawns we have $\sigma : \Delta_1 \rightsquigarrow \Delta_2$

$$\frac{\sigma : \Delta_1 \rightsquigarrow \Delta_2 \quad \Delta_2 \vdash P :: z : C}{\Delta_1 \vdash \rho[\sigma].P :: z : C}$$

Additives vs multiplicatives in BI

.. database example..

Meta-theoretical properties of πBI

- If $\Delta \vdash P :: x : A$ and $P \equiv Q$, then $\Delta \vdash Q :: x : A$

Meta-theoretical properties of πBI

- ▶ If $\Delta \vdash P :: x : A$ and $P \equiv Q$, then $\Delta \vdash Q :: x : A$
- ▶ If $\Delta \vdash P :: x : A$ and $P \longrightarrow Q$, then $\Delta \vdash Q :: x : A$

Meta-theoretical properties of πBI

- ▶ If $\Delta \vdash P :: x : A$ and $P \equiv Q$, then $\Delta \vdash Q :: x : A$
- ▶ If $\Delta \vdash P :: x : A$ and $P \longrightarrow Q$, then $\Delta \vdash Q :: x : A$
- ▶ Given a empty bunch Σ (only composed of $\emptyset_m$ and $\emptyset_a$) such that $\Sigma \vdash P :: z : A$ with $A \in \{1_m, 1_a\}$, then either
 - ▶ $P \longrightarrow -$, or
 - ▶ $P \equiv \bar{z}\langle \rangle$, or $P \equiv \rho[\emptyset].\bar{z}\langle \rangle$

Meta-theoretical properties of πBI

- ▶ If $\Delta \vdash P :: x : A$ and $P \equiv Q$, then $\Delta \vdash Q :: x : A$
- ▶ If $\Delta \vdash P :: x : A$ and $P \longrightarrow Q$, then $\Delta \vdash Q :: x : A$
- ▶ Given a empty bunch Σ (only composed of $\emptyset_m$ and $\emptyset_a$) such that $\Sigma \vdash P :: z : A$ with $A \in \{1_m, 1_a\}$, then either
 - ▶ $P \longrightarrow -$, or
 - ▶ $P \equiv \bar{z}\langle \rangle$, or $P \equiv \rho[\emptyset].\bar{z}\langle \rangle$
- ▶ If $\Delta \vdash P :: x : A$, then P is weakly normalizing

Meta-theoretical properties of πBI

- ▶ If $\Delta \vdash P :: x : A$ and $P \equiv Q$, then $\Delta \vdash Q :: x : A$
- ▶ If $\Delta \vdash P :: x : A$ and $P \longrightarrow Q$, then $\Delta \vdash Q :: x : A$
- ▶ Given a empty bunch Σ (only composed of $\emptyset_m$ and $\emptyset_a$) such that $\Sigma \vdash P :: z : A$ with $A \in \{1_m, 1_a\}$, then either
 - ▶ $P \longrightarrow -$, or
 - ▶ $P \equiv \bar{z}\langle \rangle$, or $P \equiv \rho[\emptyset].\bar{z}\langle \rangle$
- ▶ If $\Delta \vdash P :: x : A$, then P is weakly normalizing
 - ▶ $\exists S. P \longrightarrow^* S \not\rightarrow -$